Mahalo Care Platform — Build Specification v2 (built on Replit, deployed to GCP)

One configurable platform for RPM, RTM, CCM, PCM and BHI, with therapeutic templates for cardiovascular, diabetes, musculoskeletal and behavioral health. Web workspace, patient PWA, and React Native apps. Built and run on Replit during development with synthetic data only; production runs on GCP. This version replaces the earlier GCP-first specification.

0. Read this first: build on Replit, deploy to GCP

The platform is built and run on Replit with synthetic data only, and production runs on GCP. The code is written so that moving from one to the other changes configuration, not code. Sections 1 to 17 are for Claude Code and the build team. Section 18 is the deployment guide for the DevOps team. Section 19 lists the decisions still open.

	Replit (build phase)	GCP (production)
Purpose	Build, run, test, demo with synthetic data	Run live tenants with real patient data
Data	Synthetic only, always	Real patient data (PHI)
Source of truth	GitHub repository; Replit pulls from it	GitHub repository; images built by CI
Runtime	Replit processes started by one command	Docker images on Cloud Run
Database	Postgres 16 on Replit's database or an external Postgres	Cloud SQL Postgres 16
Queues	Redis (external service or Redis inside the Repl)	Memorystore Redis
Files	Replit App Storage through the storage adapter	Cloud Storage through the same adapter
Secrets	Replit Secrets	Secret Manager
AI	Anthropic API directly (synthetic data only)	Claude on Vertex AI under the Google BAA

The nine portability rules. Claude Code must follow these from the first commit. They are what make the GCP move a deployment task and not a rewrite.

1. **GitHub is the source of truth.** All work goes through Linear issues, branches and pull requests. The Repl is a workspace that pulls from `main` or a feature branch. Nothing lives only in Replit.
2. **All settings come from environment variables.** One module, `packages/config`, reads and validates them with Zod at startup and fails fast with a clear message. No other file reads `process.env`.
3. **Every outside service sits behind an adapter chosen by an environment variable.** Storage, field encryption, email, SMS, video, AI, device vendors and EHRs each have a `local` or `console` driver for Replit and a production driver for GCP. Section 4 lists the drivers.

4. **PHI guard.** The setting `DATA_CLASS` is `synthetic` or `phi`. The app refuses to start with `DATA_CLASS=phi` unless `APP_ENV=prod` and every adapter is a production driver. Replit and dev can only run `synthetic`.
5. **Docker-ready from the first commit.** Each service has a Dockerfile. Replit cannot build Docker images, so GitHub Actions builds every image on every pull request and runs a container smoke test. A broken Dockerfile fails the pull request.
6. **No Replit-only services.** No Replit Auth, Replit Database (key-value), or any Replit SDK outside the storage adapter. Authentication is Better Auth on Postgres; data is Postgres; jobs are BullMQ on Redis.
7. **Twelve-factor behavior.** Stateless processes, listen on `PORT` and `0.0.0.0`, JSON logs to stdout, graceful shutdown on SIGTERM, health endpoints, migrations as a separate command, writes only to `/tmp`.
8. **Row-level security must work on Replit.** The app connects as a dedicated non-owner role without bypass rights, and CI proves one tenant cannot read another's rows. Managed Postgres defaults often bypass row-level security, so this is tested, not assumed.
9. **Configuration is portable data.** `pnpm config:export` writes a tenant's programs, templates, rules and care-plan templates to JSON, and `pnpm config:import` loads them elsewhere. Patient data is never exported this way.

What Replit is never used for

- Real patient data, real phone numbers or real email addresses of patients.
- A production or customer-facing live service. A Replit deployment may host a demo with synthetic data only.
- Anything the DevOps team would have to reverse-engineer later. If it works only because of Replit, it is a bug.

1. Purpose, scope, non-goals

Build Mahalo Care as one configurable platform that runs RPM, RTM, CCM, PCM and BHI for any outpatient group, with therapeutic-area templates (cardiovascular, diabetes, musculoskeletal, behavioral health) layered on top. The first customer shape is a multi-site specialty group such as Groth Pain & Spine (up to 12 locations, about 10,000 unique patients a month, RTM for pain and spine, RPM for hypertension, CCM for pain plus a second condition), but nothing in the build is specific to one customer.

This document is the single source of truth for Claude Code. It is written so Claude Code can run a full SDLC (Linear issues, GitHub pull requests, CI, Replit for building, GCP for production) without asking what to build. Where a fact must be checked by a person (CMS code values, payer rules, device vendors, Replit limits), it is marked **VERIFY**.

In scope

- Clinician, care-manager and admin web workspace (React).
- Patient web app (PWA) plus React Native patient and clinician companion apps (Expo, one codebase for iOS and Android) with Bluetooth devices and HealthKit/Health Connect, all in v1 (section 10a).
- Program engine: enrollment, consent, care plans, time tracking, device-day counting, exceptions, worklists, notes.
- Billing operations engine with versioned CPT rule packs per program and a concurrency matrix across programs.
- Device ingestion (cellular device vendors via webhook first, Bluetooth via the mobile app).
- EHR integration: FHIR R4 read, per-vendor adapters (OmniMD first custom adapter), PDF export of care plans and notes.
- Secure messaging, SMS, email, in-app video.
- Governed AI (summaries, prioritization, draft notes) with full logging and human sign-off.
- Multi-tenant, multi-site, white label.
- Full SDLC: monorepo, GitHub Actions, Linear, Replit build workflow, Docker images, Terraform, Cloud Run on GCP.
- A DevOps deployment guide from Replit to GCP (section 18).

Non-goals (do not build)

- Clinical staffing or any care delivery. Mahalo is platform only.
- Automatic billing determinations or claim submission. The platform prepares evidence and reports; the customer's revenue team decides and submits.
- EHR write-back in v1. Read-only FHIR plus PDF export is the documented pattern.
- Diagnosis, medication changes or emergency response by AI.
- Research features (EDC, randomisation). Those live in Mahalo Research; a consented bridge is a later project.
- Running production or real patient data on Replit.

Success criteria for v1.0

- A new tenant with 12 locations, three programs and two therapeutic templates can be configured by an admin in one working day with no code change.
- A clinician can enroll a patient, assign a care plan, log time, review a reading exception and sign a note in under five minutes end to end.
- The month-end billing report for RPM, RTM and CCM reconciles to the time log and device-day log with zero double-counted minutes, proven by automated tests.
- A care plan and a monthly note export as a PDF that a medical records clerk can file in any EHR.
- All PHI access is in the audit log; the audit log cannot be edited or deleted by any application role.
- The same Docker images that passed CI on Replit-built code deploy to GCP with configuration changes only.

2. Product principles and UX standard

The product is exception-first: a care manager opens the app and sees the patients who need attention today, why, and the next action. Everything else is one click deeper. Old RPM and CCM tools fail because they show every reading, every task and every field at once; this product hides what is routine.

Principles (apply to every screen and API)

1. One record per patient. Readings, check-ins, medications, care plan, time, notes and messages all hang off one patient timeline. No program gets its own silo.
2. Programs are configuration. RPM, RTM, CCM, PCM and BHI are rule packs loaded from data, not branches in code.
3. Exceptions, not feeds. Worklists show prioritized exceptions with an owner. Raw data is available on the patient page, never on the home screen.
4. Time is captured as you work. Every clinical action in the app can start, stop and attribute time with one click. Manual time entry is allowed but flagged as manual in the audit trail.
5. Billing evidence is a by-product. The month-end report is generated from the same logs the clinician saw all month. There is no separate "billing module" to fill in.
6. Nothing is deleted. Clinical records are versioned and soft-closed. Audit logs are append-only.

7. Human sign-off on anything AI drafts. Drafts are visibly marked as drafts until a named user accepts them.
8. Fast on a 2019 laptop and a 4G phone. Target p75 route load under 1.5 s and interaction under 100 ms. Lists are virtualized and paginated server-side.

Visual standard (Mahalo Product UI Design System v1.0)

- Tokens: sidebar navy `#071C3B`, accent indigo `#3D36E8`, canvas `#FAFBFD`, flat cards with 1 px borders, 6 px chips, no drop shadows beyond elevation-1.
- Type: Inter for UI, Victor Serif Smooth for entity titles (patient name, program name), tabular numerals for readings and time.
- Density: one density setting per user (comfortable and compact). Tables show at most 7 columns by default; extra columns are opt-in per user and persisted.
- Color carries meaning only for status: red = needs action today, amber = due soon, green = on track, grey = inactive. Never color for decoration.
- Patient app: large type, one task per card, two-tap check-ins, a single home screen with today's tasks and a connected-device status strip. Atkinson Hyperlegible is the patient-facing face.
- Accessibility: WCAG 2.2 AA, full keyboard navigation in the workspace, screen-reader labels on every control, focus states visible.
- Empty states teach: each empty list says what will appear there and offers the first action.

Build with

- Tailwind v4 with the design tokens as CSS variables, shadcn/ui components (Radix primitives) restyled to the token set, Lucide icons.
- A `packages/ui` library holding every component, with Storybook and visual regression tests (Playwright screenshots). No component is used in an app before it exists in `packages/ui`. React Native components live in `packages/ui-native` and use the same tokens.
- Figma is the design source. Claude Code reads frames through the Figma connector when a screen is specified there; otherwise it builds from this document and the component library.
- Replit note: the Repl preview pane is for quick looks only. Visual sign-off happens in Storybook and on the dev preview, not in the Replit pane.

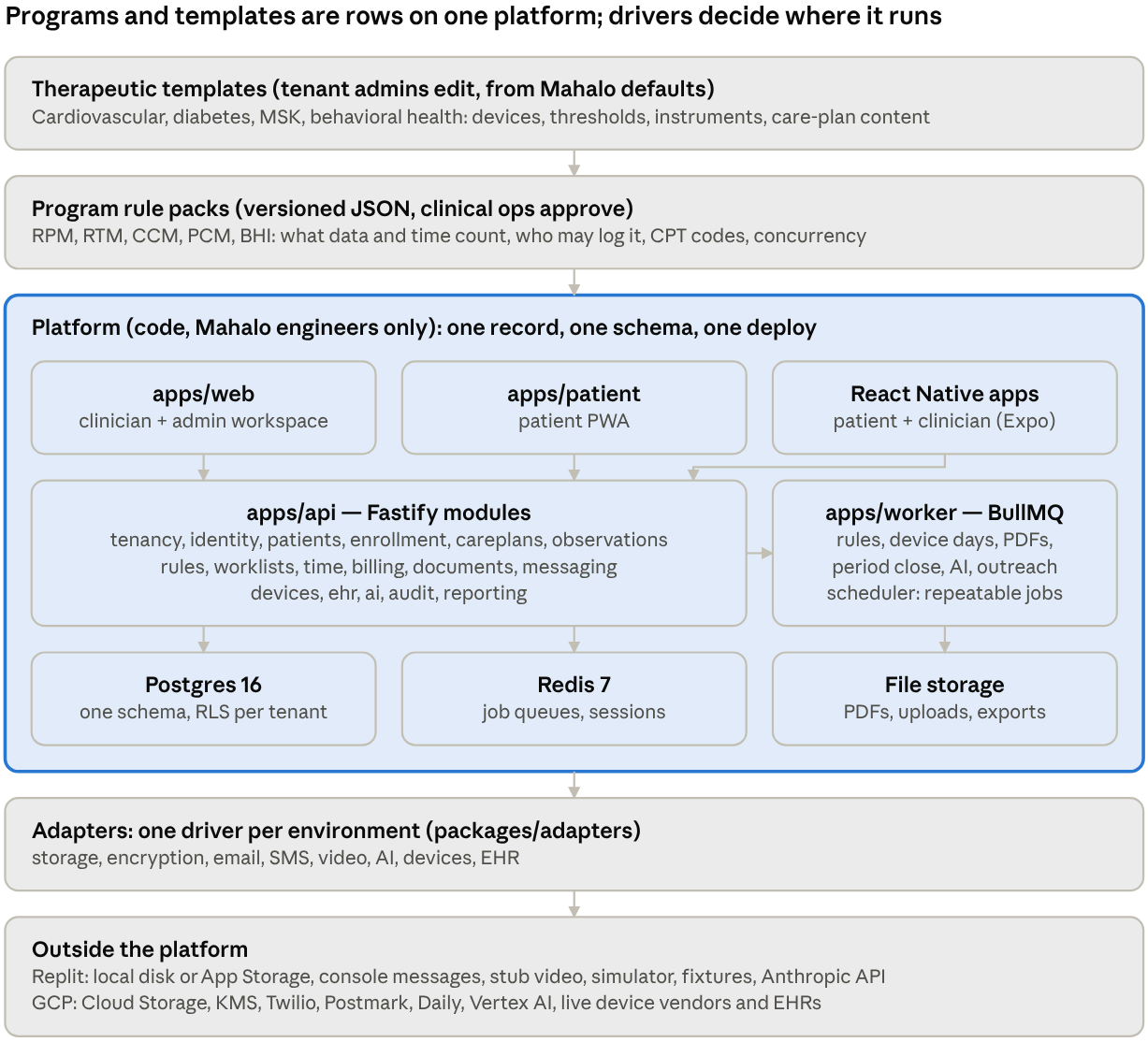
3. Architecture and the three-layer model

The system is a modular monolith on one Postgres database, split into a web app, an API, and a background worker, each built as its own Docker image from one monorepo. It is not microservices: one schema, one deploy pipeline, clear module boundaries inside the API so modules can be extracted later if load demands it. The same three processes run on Replit (as plain Node processes) and on GCP (as Cloud Run services).

Three content layers (this is the product thesis)

Layer	What it holds	Who edits it	Where it lives
Platform	Patient record, enrollment, consent, devices, ePROs, messaging, video, tasks, time log, audit, roles, multi-site, EHR adapters, AI governance	Mahalo engineers only (code)	`apps/api` modules
Program (rule pack)	RPM, RTM, CCM, PCM, BHI: what data counts, what time counts, who may log it, care-plan template, consent text, CPT code table, concurrency rules	Mahalo clinical ops, versioned and signed off	`packages/rulepacks` JSON, loaded into the `program_rulepack` table
Therapeutic template	Cardiovascular, diabetes, musculoskeletal, behavioral health: device set, thresholds, check-in instruments, education, exercise library, care-plan content	Tenant admins (from Mahalo defaults)	`care_template` tables per tenant

A program instance for a tenant is a rule pack plus a therapeutic template plus tenant settings. "RTM for spine pain" or "RPM for hypertension" is a row, not a build.



three content layers over one runtime · 3 services, 3 data stores, adapters at the edge

The two upper layers are data the platform loads; only the lowest layer is code. Clients call the API, the API hands long work to the worker through Redis, and every outside service sits behind an adapter whose driver depends on where the platform runs.

Runtime components

- `apps/web` — clinician, care-manager and admin workspace. React single-page app built to static files.
- `apps/patient` — patient PWA.
- `apps/patient-mobile` and `apps/clinician-mobile` — React Native apps (Expo) sharing `packages/mobile-core` (section 10a).

- `apps/api` — Fastify API. Modules: `tenancy`, `identity`, `patients`, `enrollment`, `careplans`, `observations` (readings, ePROs, adherence), `rules` (thresholds and exceptions), `worklists`, `time`, `billing`, `documents` (notes, PDF), `messaging`, `video`, `devices`, `ehr`, `ai`, `audit`, `reporting`.
- `apps/worker` — BullMQ workers for device webhooks, rule evaluation, reminders, billing period close, PDF rendering, EHR sync, AI jobs, outbound SMS and email. The worker also holds the scheduler: BullMQ repeatable jobs (nightly rule sweep, daily device-day roll-up, lmonth-end close) registered at start-up. There is no separate scheduler app, so cron works the same on Replit and GCP.
- `packages/db` — Drizzle schema and migrations. `packages/rulepacks` — program definitions and their JSON schema. `packages/contracts` — Zod schemas shared by API and clients, generated OpenAPI. `packages/ui` and `packages/ui-native` — component libraries. `packages/pdf` — HTML templates for care plans and notes. `packages/config` — environment parsing and the PHI guard. `packages/adapters` — the driver interfaces and the local and production drivers (section 4).

Request path

Browser or app → `web` static files and `api` → Postgres and Redis. On GCP this goes through a load balancer with Cloud Armor; on Replit it goes through the Repl's public URL. Webhooks from device vendors and EHRs hit `api/webhooks/*`, are verified, written to an inbox table, and acknowledged; a worker processes the inbox. No external call is made inside a request handler except auth checks.

Data flow for one reading

1. Vendor webhook arrives, signature verified, raw payload stored in `ingest_inbox`.
2. Worker normalizes it to an `observation` row with source, device, unit, measured-at and received-at.
3. The same worker updates `device_day` for that patient, program and calendar date.
4. Rule evaluation runs for the patient's active programs; an `exception` row is created or updated (deduplicated by rule, patient and 24-hour window).
5. The exception appears in the owning role's worklist; a notification goes out if the rule says so.
6. The clinician reviews, acts, and the time taken is logged against the exception and the program.

Boundaries that must hold

- Only the `billing` module reads `time_entry` and `device_day` for money purposes; it never writes them.
- Only the `audit` module writes `audit_log`; every other module calls it through one function.
- Rule packs are read-only at runtime; a new version is a new row with a start date.
- The AI module never writes clinical fields directly; it writes `ai_draft` rows that a human accepts.
- No module imports a vendor SDK directly; it asks `packages/adapters` for a driver.

4. Technology stack, Replit build environment, GCP target

The stack is TypeScript end to end: React and Vite on the front, Fastify and Drizzle on Node 22 behind, Postgres 16, Redis 7, packaged as Docker images. Replit runs it during the build; Cloud Run runs it in production. Pin every version in `package.json`, `.tool-versions` and the Dockerfiles; upgrade on a schedule, not ad hoc.

Stack

Concern	Choice	Why this one
Language and runtime	TypeScript 5.x strict, Node 22 LTS, pnpm workspaces, Turborepo	One language, shared types, fast CI caching
Web	React 19, Vite, TanStack Router, TanStack Query, TanStack Table (virtualized), react-hook-form + Zod	Type-safe routes and data; no server rendering needed for a logged-in app
Styling	Tailwind v4, shadcn/ui on Radix, Lucide icons, Storybook	Matches Design System v1.0 tokens, accessible primitives
Mobile	Expo (pinned SDK), React Native, expo-router, NativeWind	One codebase for iOS and Android (section 10a)
API	Fastify 5, `@fastify/type-provider-zod`, generated OpenAPI 3.1, `openapi-typescript` client	Fast, schema-first, one source of truth for request and response shapes
ORM and DB	Drizzle ORM + drizzle-kit migrations, Postgres 16	SQL you can read, migrations in git, row-level security

Concern	Choice	Why this one
Auth	Better Auth (sessions, TOTP MFA, passkeys, organizations and roles), OIDC for Google Workspace and Microsoft Entra; SAML later through an adapter if an enterprise asks	Self-hosted on Postgres, no Replit or cloud lock-in, PHI stays in the tenant boundary
Jobs	BullMQ on Redis 7, repeatable jobs for cron	Simple, retryable, observable, identical on Replit and GCP
PDF	Playwright (Chromium) in `apps/worker`, HTML templates from `packages/pdf`	Pixel-exact, same CSS as the app
Observability	pino JSON logs to stdout, OpenTelemetry (optional exporter), Sentry with a PHI scrubber	Cloud Logging reads stdout on GCP with no code change
Testing	Vitest, Supertest, Playwright, Storybook test runner, Maestro for mobile; integration database by Testcontainers in CI or by URL on Replit	Same toolchain as the published Mahalo SDLC
Infra as code	Terraform in `infra/`, deploy by GitHub Actions with Workload Identity Federation	Reviewable, repeatable, no long-lived keys

Adapters and drivers (`packages/adapters`)

Each row is an interface with two or more drivers. An environment variable picks the driver. Application code only ever calls the interface.

Concern	Interface	Replit and dev driver	GCP production driver	Variable
Files	`put`, `get`, `delete`, `signedUrl`	`local` (disk under `./.data`, never in git) or `replit` (App Storage)	`gcs` (Cloud Storage, CMEK, 15-minute signed URLs)	`STORAGE_DRIVER`
Field encryption	`encrypt`, `decrypt` with key id	`local` (AES-256-GCM, key from a Replit Secret)	`gcp-kms` (envelope encryption)	`CRYPTO_DRIVER`
Email	`send(template, to, data)`	`console` (writes to the log, sends nothing)	`postmark` or `sendgrid` under a BAA	`EMAIL_DRIVER`
SMS	`send(template, to, data)`	`console`, or `twilio` with test credentials and verified team numbers	`twilio` under a BAA	`SMS_DRIVER`
Video	`createRoom`, `joinToken`	`stub` (a fake room page) or `daily` with a test domain	`daily` under a BAA	`VIDEO_DRIVER`
AI	`complete(promptId, inputs)`	`anthropic` (direct API, synthetic data only) or `mock`	`vertex` (Claude on Vertex AI under the Google BAA)	`AI_DRIVER`

Concern	Interface	Replit and dev driver	GCP production driver	Variable
Devices	`verify`, `parse`, `health` per vendor	`simulator` (`tools/device-sim` posts signed fake webhooks) and vendor sandbox accounts	Live vendor accounts	`DEVICE_DRIVERS`
EHR	FHIR client and per-vendor adapters	`fixture` (recorded FHIR bundles) and public FHIR sandboxes	Live connections per tenant	`EHR_DRIVER`

Environment variable contract (`packages/config`)

The config module parses these with Zod, fails fast with a readable error, and is the only code that reads `process.env`.

Variable	On Replit	On GCP	Notes
`APP_ENV`	`replit`	`dev`, `stg` or `prod`	Selects allowed drivers
`DATA_CLASS`	`synthetic`	`synthetic` in dev and stg, `phi` only in prod	The PHI guard (section 0, rule 4)
`NODE_ENV`	`development`	`production`	
`HOST`, `PORT`	`0.0.0.0` and one port per process	`0.0.0.0` and the `PORT` Cloud Run sets	Never bind to localhost
`APP_BASE_URL`, `API_BASE_URL`	The Repl's public URL	The tenant's HTTPS URLs	Used in links, cookies, CORS

Variable	On Replit	On GCP	Notes
`DATABASE_URL`	App role connection (no bypass rights)	Cloud SQL private connection, app role	Used by api and worker
`DATABASE_MIGRATE_URL`	Owner role connection	Owner role, used only by the migration job	Never given to api or worker
`REDIS_URL`	External Redis (TLS) or the in-Repl Redis	Memorystore	Must allow `noeviction` for BullMQ
`STORAGE_DRIVER`, `STORAGE_BUCKET`	`replit` or `local`	`gcs` and the bucket name	
`CRYPTO_DRIVER` and `FIELD_ENCRYPTION_KEY` or `KMS_KEY_NAME`	`local` and a Replit Secret	`gcp-kms` and the key resource name	
`AUTH_SECRET`	Replit Secret	Secret Manager	Rotated per environment
`TENANT_RESOLUTION`	`membership`	`subdomain`	Section 14
`EMAIL_DRIVER`, `SMS_DRIVER`, `VIDEO_DRIVER`, `AI_DRIVER`, `EHR_DRIVER`, `DEVICE_DRIVERS`	Local or sandbox drivers	Production drivers	See adapter table
Provider keys	Test keys in Replit Secrets	Secret Manager	`ANTHROPIC_API_KEY` is allowed only when `APP_ENV` is `replit` or `dev`
`CHROMIUM_PATH`	Path from the Nix package	Set in the worker image	For PDF rendering
`LOG_LEVEL`, `SENTRY_DSN`, `OTEL_EXPORTER_OTLP_ENDPOINT`	Optional	Set	
`TZ`	`UTC`	`UTC`	All business time conversion is explicit

Replit build environment

- **Workspace.** One private Repl for the monorepo, connected to the GitHub repository, named members only, two-factor on every account. No public Repls. Secrets hold synthetic and test credentials only; no production credential ever enters Replit.
- **Builder.** Claude Code is the default builder and runs in the Replit shell or on a developer's machine, pushing branches to GitHub. Replit Agent may be used on a feature branch for UI work; it reads `replit.md`, which mirrors the rules in `CLAUDE.md`. Either way, code reaches `main` only through a reviewed pull request (section 15).
- **Files that configure Replit.** `.replit` (run command `pnpm dev`, port mappings, deployment target), `replit.nix` (Node 22, pnpm, Chromium and its libraries, a Postgres client, optionally Redis), and `replit.md` (rules for the agent). These are in git so any developer can recreate the Repl.
- **One command.** `pnpm dev` starts the API, the web app, the patient PWA and the worker together. `pnpm setup` installs, migrates, seeds a demo tenant with three programs and 50 synthetic patients, and starts everything.
- **One origin.** The Vite dev server on the Repl's external port proxies `/api` and `/webhooks` to the API. On GCP the load balancer does the same with path rules. This keeps cookies, CORS and webhook URLs identical in both places. **VERIFY** which ports Replit exposes externally and map them in `.replit`.
- **Database.** Use Replit's built-in Postgres or an external Postgres 16 (Neon, Supabase or similar). **VERIFY** the major version matches production. Create a dedicated app role with `CREATE ROLE app_rw LOGIN` that is not the owner and does not have bypass-RLS rights, give it only the needed grants, and run migrations with the owner role through `DATABASE_MIGRATE_URL`. Set `FORCE ROW LEVEL SECURITY` on every tenant table. Managed Postgres default roles often bypass row-level security, so a CI test connects as the app role and proves tenant A cannot read tenant B.
- **Redis.** Replit has no managed Redis. Use an external Redis with TLS (Upstash, Redis Cloud) or run Redis from Nix inside the Repl. BullMQ needs the `noeviction` memory policy; **VERIFY** the provider allows it.
- **PDF rendering.** The worker needs Chromium. Install it through Nix, set `CHROMIUM_PATH`, and prove it renders a care plan in milestone 0. In Docker the worker uses the Playwright base image pinned to the same Playwright version as the lockfile.
- **No Docker on Replit.** Replit cannot build or run Docker images. So Dockerfiles are validated in GitHub Actions on every pull request, and integration tests run against the Repl's database with `TEST_DB_MODE=url` (a throwaway schema created and

dropped per run), while CI uses `TEST_DB_MODE=testcontainers`.

- **What runs where.** Replit runs unit tests, API integration tests, the app and quick smoke checks. Full Playwright suites, Maestro mobile flows, image builds, security scans and migration up-down-up checks run in GitHub Actions, because Replit's memory and CPU are limited.
- **Mobile.** The Expo Metro server can run in the Repl and testers open the app in Expo Go for UI work. Bluetooth, HealthKit and Health Connect need native modules that Expo Go does not include, so those use EAS development builds installed on real devices. **VERIFY** the Metro tunnel works from the Repl.
- **Replit Deployments.** A Replit deployment may host a demo for the team or a prospect with synthetic data only. It is never given real patient data or live vendor credentials.

GCP target (production)

- Projects per environment: `mahalo-care-dev`, `mahalo-care-stg`, `mahalo-care-prod`. Region `us-east4` for the US tenant, with region as a tenant attribute.
- Cloud Run services: `web` (static files served by an nginx image), `api`, `worker`. The worker has CPU always allocated and at least one instance, because it runs queues and repeatable jobs.
- Cloud SQL Postgres 16 (high availability, private IP, 35-day backups, point-in-time recovery, CMEK). Memorystore Redis 7. Cloud Storage buckets with CMEK for documents, exports and backups.
- Global HTTPS load balancer with managed certificates and Cloud Armor (OWASP rules, rate limits on `/auth/*` and `/webhooks/*`), with path routing so `/api/*` and `/webhooks/*` go to `api`.
- Secret Manager for secrets, Cloud KMS for keys, Artifact Registry for images, Cloud Logging and Monitoring, audit logs exported to a locked bucket.
- Images are built once in CI, signed, scanned, and promoted dev → staging → prod by digest, never rebuilt.
- Only HIPAA-covered GCP services are used, and the Google Cloud BAA must be signed on the organization before any real data exists (**VERIFY** the BAA is executed).

The full step-by-step deployment is in section 18.

Environments

Env	Purpose	Data	Deploy trigger
replit	Build, run, test, internal demos	Synthetic only	Pull from GitHub; run `pnpm dev`
ci	Every pull request: checks, image build, smoke	Synthetic fixtures	Pull request
dev (GCP)	First created for the dry run, then integration and preview	Synthetic only	Merge to `main`
staging (GCP)	Customer demos, UAT, release candidate	Synthetic or de-identified, never real PHI	Tag `rc-*`
prod (GCP)	Live tenants	PHI	Tag `v*` after staging sign-off and manual approval

5. Data model (Drizzle schema)

One Postgres schema, every tenant-owned table carries `tenant_id`, and Postgres row-level security enforces tenant isolation under the application role. Claude Code writes these as Drizzle tables in `packages/db/src/schema/<module>.ts`, one file per module below, with a migration per PR.

Conventions (enforced by lint and a schema test)

- Primary keys are UUID v7 so rows sort by time. Generate them in the application (a small `uuidv7()` helper) so the schema does not depend on a database extension that may differ between Replit's Postgres and Cloud SQL.
- Every table has `created_at`, `updated_at` (timestamptz, UTC) and `created_by` (user id or `system`).
- Every tenant table has `tenant_id` not null; composite foreign keys include `tenant_id` so a row can never reference another tenant.
- RLS policy on every tenant table: `tenant_id = current_setting('app.tenant_id')::uuid`, with `FORCE ROW LEVEL SECURITY` on. The API sets this per request inside a transaction. The application connects as

`app_rw`, a role that is not the table owner and has no bypass-RLS right. A separate `platform_admin` role bypasses RLS for support, and every such use is audited. Migrations run as the owner role only.

- No hard deletes on clinical tables. Use `status` columns and `closed_at`. A `deleted_at` column exists only on `message`, `notification` and `task`.
- JSON columns (`jsonb`) are validated by a Zod schema in `packages/contracts` on write; the Zod schema name is recorded in a column comment.
- Times a patient acts on (readings, check-ins, device days) are stored in UTC with the patient's IANA timezone on the patient row; the device-day date is computed in the patient's timezone and stored as a `date`.
- Money is never stored; the platform stores units, minutes and days. Rates belong to the customer.
- Use only Postgres features available on both Replit's database and Cloud SQL (plain SQL, `jsonb`, `pg_trgm`, exclusion constraints with `btree_gist`). Migrations create the extensions they need with `CREATE EXTENSION IF NOT EXISTS`; **VERIFY** both are allowed on the Replit database in milestone 0.

Tenancy and identity (`tenancy.ts`, `identity.ts`)

Table	Key columns	Rules
`tenant`	`slug`, `name`, `region`, `branding` jsonb, `settings` jsonb, `status`	One row per customer. Region decides storage and hosting.
`location`	`tenant_id`, `name`, `address`, `timezone`, `place_of_service_code`, `group_npi`, `status`	A physical office. Place of service and group NPI appear on billing lines.
`user`	`tenant_id`, `email`, `name`, `credential` enum (MD, DO, NP, PA, RN, LPN, MA, PT, OT, SLP, LCSW, psych_consultant, admin, billing), `individual_npi`, `status`	Credential drives which time counts for which program. Auth tables come from Better Auth.

Table	Key columns	Rules
`role`	`tenant_id`, `key`, `permissions` jsonb	Defaults seeded: `clinical_admin`, `billing_provider`, `care_manager`, `therapist`, `front_desk`, `billing`, `tenant_admin`, `read_only`.
`user_role`	`user_id`, `role_id`, `location_id` nullable	Null location = all locations.
`user_location`	`user_id`, `location_id`, `is_primary`	Where a user usually works; used for default filters only.

Patients (`patients.ts`)

Table	Key columns	Rules
`patient`	`tenant_id`, `mrn`, `given_name`, `family_name`, `dob`, `sex`, `phone`, `email`, `language`, `timezone`, `home_location_id`, `status`, `comm_prefs` jsonb	Exactly one home location, changeable with an audited reason. Cross-site access is by role, not by moving the row.
`patient_identifier`	`patient_id`, `system`, `value`	EHR ids, payer member ids. Unique on `(tenant_id, system, value)`.
`patient_condition`	`patient_id`, `icd10`, `display`, `onset_date`, `status`, `is_chronic`, `source` (ehr, manual)	CCM eligibility reads `is_chronic` count; PCM reads the one condition on the enrollment.
`patient_contact`	`patient_id`, `relationship`, `name`, `phone`, `can_receive_phi`	Caregivers.
`coverage`	`patient_id`, `payer_kind` (medicare, medicare_advantage, commercial, medicaid, self), `payer_name`, `member_id`, `effective`, `end`	Used to pick payer overrides and to warn on cost-sharing consent.

Programs, enrollment, consent (`programs.ts`)

Table	Key columns	Rules
`program_rulepack`	`key` (rpm, rtm, ccm, pcm, bhi), `version`, `effective_from`, `effective_to`, `definition` jsonb, `checksum`, `approved_by`, `approved_at`	Global, not tenant-owned. Loaded from `packages/rulepacks`. Immutable once approved.
`care_template`	`tenant_id` nullable (null = Mahalo default), `area_key` (cardiovascular, diabetes, msk, behavioral), `name`, `version`, `definition` jsonb	Devices, thresholds, instruments, education, exercise library, care-plan template reference.
`program`	`tenant_id`, `rulepack_id`, `care_template_id`, `name`, `settings` jsonb, `default_billing_provider_id`, `status`	A configured program instance, e.g. "RTM – Spine".
`enrollment`	`tenant_id`, `patient_id`, `program_id`, `status` (pending_consent, active, paused, discharged), `start_date`, `end_date`, `billing_provider_id`, `ordering_provider_id`, `home_location_id`, `primary_condition_id`, `consent_id`, `discharge_reason`	One per patient per program. Concurrency rules in section 7 are checked on activate.
`consent`	`tenant_id`, `patient_id`, `enrollment_id` nullable, `kind` (program, device_sharing, communication, video), `text_version`, `method` (verbal, written, electronic), `captured_by`, `captured_at`, `document_id`, `revoked_at`	Verbal consent is allowed for CCM but must be documented; the rule pack says which kinds are required before activation.

Care plans (`careplans.ts`)

Table	Key columns	Rules
`careplan_template`	`tenant_id` nullable, `program_key`, `name`, `version`, `sections` jsonb	Sections carry `required: true` elements; a plan cannot be signed with a required element empty.
`care_plan`	`enrollment_id`, `template_id`, `version`, `status` (draft, active, superseded, closed), `content` jsonb, `effective_from`, `authored_by`, `signed_by`, `signed_at`, `prior_version_id`	New version on every change after signing; old versions stay readable.
`care_plan_goal`	`care_plan_id`, `title`, `measure`, `target`, `baseline`, `status`, `due_date`	
`care_plan_item`	`care_plan_id`, `kind` (medication, exercise, reading, epro, education, appointment, custom), `title`, `schedule_rrule`, `window_minutes`, `details` jsonb, `active`	Generates patient tasks.
`care_plan_review`	`care_plan_id`, `reviewed_by`, `reviewed_at`, `summary`, `time_entry_id`	A review is a billable activity under CCM and PCM.

Observations and devices (`observations.ts`, `devices.ts`)

Table	Key columns	Rules
`device_catalog`	`vendor`, `model`, `kind` (bp_cuff, scale, glucometer, pulse_ox, spirometer, inhaler, motion_sensor, wearable), `fda_cleared`, `clearance_ref`, `connectivity` (cellular, ble, api), `data_types` jsonb	Global list; `fda_cleared` decides whether readings may count toward RPM device days.

Table	Key columns	Rules
`device`	`tenant_id`, `catalog_id`, `serial`, `vendor_device_id`, `status`, `last_seen_at`	
`device_assignment`	`device_id`, `patient_id`, `enrollment_id`, `assigned_at`, `returned_at`, `shipped_at`, `setup_completed_at`, `setup_by`, `setup_time_entry_id`	Setup completion is the evidence for setup codes.
`ingest_inbox`	`source`, `received_at`, `headers` jsonb, `payload` jsonb, `signature_valid`, `status`, `error`, `processed_at`	Raw webhooks. Kept 2 years.
`observation`	`tenant_id`, `patient_id`, `enrollment_id`, `loinc`, `display`, `value_num`, `value_text`, `unit`, `measured_at`, `received_at`, `source` (device, patient_reported, wearable, ehr, clinician), `device_id`, `inbox_id`, `is_valid`, `invalid_reason`	Unique on `(device_id, loinc, measured_at)` to drop vendor replays. Invalid readings stay but never count.
`device_day`	`enrollment_id`, `program_id`, `day` date, `source_kind`, `observation_count`, `counted`, `reason`	Unique on `(enrollment_id, day)`. Rebuilt by a nightly job and on every new observation.
`epro_instrument`	`key` (phq9, gad7, pain_nrs, odi, pgic, promis_pf, custom), `version`, `items` jsonb, `scoring` jsonb, `license_note`	Global; custom instruments are tenant-owned copies. Check each instrument's license before shipping its text (**VERIFY**).
`epro_response`	`enrollment_id`, `instrument_id`, `task_id`, `answers` jsonb, `score`, `band`, `completed_at`, `source`	Score and band computed server-side.

Table	Key columns	Rules
`adherence_event`	`enrollment_id`, `care_plan_item_id`, `due_at`, `status` (done, missed, skipped), `reported_at`, `barrier_code`, `barrier_text`, `source`	Barrier codes: too_painful, side_effects, no_time, forgot, unclear, other.
`medication`	`patient_id`, `rxnorm`, `name`, `dose`, `route`, `frequency_text`, `schedule_rrule`, `source`, `status`	Adherence flows through `adherence_event`.

Rules, exceptions, tasks (`rules.ts`, `work.ts`)

Table	Key columns	Rules
`rule`	`tenant_id`, `program_id` nullable, `care_template_id` nullable, `key`, `kind` (threshold, trend, missing_data, adherence, score, time_budget, device_days_at_risk), `definition` jsonb, `severity`, `owner_role_id`, `dedupe_minutes`, `notify` jsonb, `version`, `active`	Definition grammar in section 9.
`exception`	`tenant_id`, `patient_id`, `enrollment_id`, `rule_id`, `severity`, `status` (open, acknowledged, resolved, dismissed), `opened_at`, `last_signal_at`, `signal_count`, `assigned_to`, `resolved_by`, `resolved_at`, `resolution_code`, `note_id`	One open exception per rule and patient; new signals increment the count instead of creating a new row.
`task`	`tenant_id`, `patient_id` nullable, `enrollment_id` nullable, `audience` (patient, staff), `kind`, `title`, `due_at`, `assigned_role_id`, `assigned_to`, `status`, `source` (careplan, rule, manual, system), `payload` jsonb, `completed_at`	Patient tasks and staff tasks share one table.

Time, interactions, notes, documents (`time.ts`, `documents.ts`)

Table	Key columns	Rules
`time_entry`	`tenant_id`, `patient_id`, `enrollment_id`, `program_id`, `user_id`, `user_credential_snapshot`, `started_at`, `ended_at`, `minutes` (generated), `activity_code`, `is_interactive`, `interaction_id`, `exception_id`, `task_id`, `note_id`, `source` (timer, manual, ai_suggested), `status` (draft, committed, voided), `billing_period_id`	Exclusion constraint: no two committed entries for one user overlap in time. One entry belongs to exactly one program. Voiding keeps the row and requires a reason.
`interaction`	`tenant_id`, `patient_id`, `enrollment_id`, `kind` (phone, video, sms, secure_message, in_person), `direction`, `started_at`, `ended_at`, `participants` jsonb, `outcome`, `is_live_two_way`	`is_live_two_way` is the RPM and RTM interactive-communication evidence.
`note`	`tenant_id`, `patient_id`, `enrollment_id` nullable, `kind` (monthly_summary, encounter, care_plan_review, exception_resolution, free), `template_id`, `content` jsonb, `status` (draft, signed, amended), `author_id`, `signed_by`, `signed_at`, `prior_version_id`, `ai_draft_id`	Signed notes are immutable; amendments create a new version linked to the prior.
`document`	`tenant_id`, `patient_id` nullable, `kind` (care_plan_pdf, note_pdf, consent, billing_report, export), `storage_uri`, `sha256`, `generated_at`, `generated_by`, `source_refs` jsonb	SHA-256 proves the PDF matches what was signed. `storage_uri` is driver-neutral (`storage://tenant/...`), so it resolves on Replit or GCP.

Billing (`billing.ts`)

Table	Key columns	Rules
`billing_period`	`enrollment_id`, `program_id`, `period_kind` (calendar_month, rolling_30), `period_start`, `period_end`, `status` (open, closed, reopened), `closed_at`, `closed_by`, `rulepack_version`	Rolling 30-day periods start at the first counted device day; calendar months start on the 1st in the tenant timezone.
`billing_evidence`	`billing_period_id`, `cpt_code`, `met`, `quantity`, `minutes_counted`, `device_days_counted`, `live_interactions`, `requirements` jsonb, `blockers` jsonb, `computed_at`	Recomputed on every relevant write; the month-end job freezes it.
`billing_line`	`billing_period_id`, `cpt_code`, `units`, `rendering_provider_id`, `billing_provider_id`, `location_id`, `dx_codes` jsonb, `status` (candidate, approved, exported, rejected), `decided_by`, `decided_at`, `reason`	Candidates are generated; humans approve.
`billing_export`	`tenant_id`, `month`, `format` (csv, xlsx), `document_id`, `line_count`, `exported_by`	837 claim files are out of scope for v1.
`payer_override`	`tenant_id`, `payer_kind`, `payer_name`, `cpt_code`, `override` jsonb	Lets a tenant tighten (never loosen) a rule for a payer.

Messaging, EHR, AI, audit (`messaging.ts`, `ehr.ts`, `ai.ts`, `audit.ts`)

Table	Key columns	Rules
`thread`, `message`	`patient_id`, `participants`, `sender_user_id` or `sender_patient_id`, `body`, `attachments` jsonb, `sent_at`, `read_at`	Secure messages only; SMS is a notification channel, not a thread.

Table	Key columns	Rules
`notification`	`recipient_user_id` or `recipient_patient_id`, `channel` (push, sms, email, in_app), `template_key`, `payload` jsonb, `status`, `sent_at`, `provider_message_id`	SMS and email bodies never contain clinical values, only a prompt to open the app.
`video_session`	`interaction_id`, `provider`, `room_id`, `started_at`, `ended_at`, `participants` jsonb	No recordings in v1.
`ehr_connection`	`tenant_id`, `vendor`, `kind` (fhir_r4, custom), `base_url`, `secret_ref`, `capabilities` jsonb, `status`, `last_sync_at`	One per tenant per EHR. `secret_ref` names a secret; the value is never in the table.
`patient_ehr_link`	`patient_id`, `connection_id`, `external_patient_id`, `last_synced_at`	
`ehr_resource_cache`	`patient_id`, `resource_type`, `resource_id`, `fhir` jsonb, `fetched_at`	Read-only copy of FHIR resources shown in the app.
`ai_draft`	`tenant_id`, `patient_id`, `kind` (summary, note_draft, prioritization, message_draft, time_suggestion), `inputs` jsonb (ids only), `prompt_version`, `model`, `output`, `status` (pending, accepted, edited, rejected), `decided_by`, `decided_at`, `final_text`	The only place AI output lands.
`ai_call_log`	`tenant_id`, `draft_id`, `provider`, `model`, `prompt_hash`, `input_tokens`, `output_tokens`, `latency_ms`, `at`	No PHI.

Table	Key columns	Rules
`audit_log`	`tenant_id`, `actor_kind` (user, patient, system, platform_admin), `actor_id`, `action`, `entity_type`, `entity_id`, `diff` jsonb, `ip`, `user_agent`, `request_id`, `at`	Append-only: the application role has INSERT only; a trigger rejects UPDATE and DELETE; partitioned by month; chart opens are logged as `read`.

Indexes to create in the first migration: `observation (enrollment_id, measured_at desc)`, `device_day (enrollment_id, day)`, `exception (tenant_id, status, assigned_to, severity)`, `time_entry (enrollment_id, started_at)`, `time_entry (user_id, started_at)`, `task (tenant_id, audience, status, due_at)`, `patient (tenant_id, home_location_id)`, `audit_log (tenant_id, entity_type, entity_id, at)`, plus a GIN index on `patient` name trigram for search.

6. Program rule packs: RPM, RTM, CCM, PCM, BHI

Each program is a versioned JSON document in `packages/rulepacks/<key>/<version>.json`, validated against one Zod schema, approved by Mahalo clinical ops, and loaded into `program_rulepack`. Code values below reflect the 2026 CMS fee schedule as understood on 7 November 2026 and are marked **VERIFY**: Claude Code must not change a code or a threshold without a Linear issue that links the CMS source.

Rule pack grammar (`packages/rulepacks/schema.ts`)

```
{
  "key": "rpm",
  "version": "2026.1",
  "effective_from": "2026-01-01",
  "eligibility": {
    "min_chronic_conditions": 1,
    "condition_min_duration_months": null,
    "initiating_visit_required_within_months": null,
    "required_consents": ["program", "device_sharing"],
    "device_requirement": "fda_cleared"
  },
  "data_counted": ["device"],
  "time_rules": {
```

```
    "counts_for_management": {
      "clinical_staff_general_supervision": ["RN", "LPN", "MA", "NP", "PA",
"MD", "DO"],
      "practitioner_only": []
    },
    "activities": ["review_data", "phone_call", "video_visit",
"care_plan_update", "care_coordination", "patient_education",
"documentation"],
    "rounding": "none",
    "exclusive_with_programs": ["rtm"]
  },
  "codes": [
    { "cpt": "99453", "kind": "setup", "period": "episode", "max_units": 1,
      "requires": { "device_setup_completed": true, "education_documented":
true } },
    { "cpt": "99454", "kind": "supply", "period": "rolling_30", "max_units":
1,
      "requires": { "device_days_min": 16 }, "excludes": ["99445"] },
    { "cpt": "99445", "kind": "supply", "period": "rolling_30", "max_units":
1,
      "requires": { "device_days_min": 2, "device_days_max": 15 },
"excludes": ["99454"] },
    { "cpt": "99470", "kind": "management", "period": "calendar_month",
"max_units": 1,
      "requires": { "minutes_min": 10, "minutes_max": 19,
"live_interactive_min": 1 }, "excludes": ["99457"] },
    { "cpt": "99457", "kind": "management", "period": "calendar_month",
"max_units": 1,
      "requires": { "minutes_min": 20, "live_interactive_min": 1 } },
    { "cpt": "99458", "kind": "addon", "parent": "99457", "period":
"calendar_month", "max_units": 2,
      "requires": { "minutes_per_unit": 20 } }
  ],
  "concurrency": [
    { "with": "rtm", "rule": "exclusive_same_period" },
    { "with": "ccm", "rule": "allowed_no_shared_minutes" },
    { "with": "pcm", "rule": "allowed_no_shared_minutes" },
    { "with": "bhi", "rule": "allowed_no_shared_minutes" }
  ],
  "careplan_template_key": "rpm_default",
  "required_instruments": []
}
```

RPM — remote physiologic monitoring (VERIFY)

Code	Kind	Period	Requirement the engine checks
99453	Setup and education	Once per episode	Device assignment with `setup_completed_at`; education task completed
99454	Device supply	Rolling 30 days	16 or more counted device days from an FDA-cleared device
99445	Device supply, short	Rolling 30 days	2 to 15 counted device days; never with 99454 in the same period
99470	Management, short	Calendar month	10 to 19 committed minutes and one live two-way interaction
99457	Management	Calendar month	20 or more committed minutes and one live two-way interaction
99458	Management add-on	Calendar month	Each further full 20 minutes, max 2 units

Data counted: device readings only. Patient-reported values never count toward device days. Only one practitioner may bill RPM for a patient in a period; the engine blocks a second active RPM enrollment for the same patient.

RTM — remote therapeutic monitoring (VERIFY)

Code	Kind	Period	Requirement the engine checks
98975	Setup and education	Once per episode	Device or app setup completed; education documented
98977	Device supply, musculoskeletal	Rolling 30 days	16 or more counted days of MSK therapeutic data
98985	Device supply, MSK short	Rolling 30 days	2 to 15 counted days; never with 98977
98976	Device supply, respiratory	Rolling 30 days	16 or more counted days of respiratory data

Code	Kind	Period	Requirement the engine checks
98984	Device supply, respiratory short	Rolling 30 days	2 to 15 counted days; never with 98976
98979	Management, short	Calendar month	10 to 19 committed minutes and one live interaction
98980	Management	Calendar month	20 or more committed minutes and one live interaction
98981	Management add-on	Calendar month	Each further full 20 minutes

Data counted: patient-reported adherence, pain and function check-ins, and supported therapeutic device or app data, each tagged with its source. The template declares which MSK or respiratory data streams count for supply days. Eligible billers include PT, OT and SLP under therapy plan-of-care rules; the rule pack lists credentials and the required modifier is a tenant setting (**VERIFY** GP, GO, GN modifier handling with the customer's billing team).

CCM — chronic care management (VERIFY)

Code	Kind	Period	Requirement the engine checks
99490	Clinical staff, first 20 min	Calendar month	20 or more clinical-staff minutes, active comprehensive care plan, consent on file
99439	Clinical staff add-on	Calendar month	Each further full 20 minutes, max 2 units
99491	Practitioner, first 30 min	Calendar month	30 or more practitioner minutes (MD, DO, NP, PA)
99437	Practitioner add-on	Calendar month	Each further full 30 practitioner minutes
99487	Complex CCM, first 60 min	Calendar month	60 or more clinical-staff minutes and `complexity = moderate_or_high` on the care plan
99489	Complex CCM add-on	Calendar month	Each further full 30 minutes

Eligibility: two or more chronic conditions expected to last at least 12 months or until death, documented consent with cost-sharing explained, initiating visit within the past 12 months for new patients (tenant setting). Only one practitioner may bill CCM for a patient in a month. 99490 and 99491 are not both billed in the same month. APCM codes (G0556 to G0558) are out of scope for v1.

PCM — principal care management (VERIFY)

Code	Kind	Period	Requirement the engine checks
99424	Practitioner, first 30 min	Calendar month	30 or more practitioner minutes, one qualifying condition on the enrollment, disease-specific care plan active
99425	Practitioner add-on	Calendar month	Each further full 30 minutes
99426	Clinical staff, first 30 min	Calendar month	30 or more clinical-staff minutes
99427	Clinical staff add-on	Calendar month	Each further full 30 minutes

Eligibility: one complex chronic condition expected to last at least 3 months that places the patient at significant risk. The enrollment stores the condition. PCM and CCM for the same patient in the same month by the same practitioner is blocked.

BHI — behavioral health integration (VERIFY)

Code	Kind	Period	Requirement the engine checks
99484	General BHI	Calendar month	20 or more clinical-staff minutes, initial assessment done, a validated scale (PHQ-9, GAD-7 or configured) scored in the month, care plan active
99492	CoCM, first month	Calendar month	70 or more behavioral care-manager minutes, psychiatric consultant assigned, registry entry, scale scored
99493	CoCM, subsequent month	Calendar month	60 or more minutes, same requirements

Code	Kind	Period	Requirement the engine checks
99494	CoCM add-on	Calendar month	Each further full 30 minutes
G2214	CoCM, short	Calendar month	30 or more minutes in an initial or subsequent month; never with 99492 or 99493

CoCM requires two roles on the enrollment: `behavioral_care_manager` and `psych_consultant`. The registry is a saved view over active CoCM enrollments with last score, last consultant review and days since contact. General BHI and CCM may be billed in the same month without shared minutes.

Concurrency matrix (one patient, one period) (VERIFY)

	RPM	RTM	CCM	PCM	BHI
RPM	—	Blocked	Allowed, separate minutes	Allowed, separate minutes	Allowed, separate minutes
RTM	Blocked	—	Allowed, separate minutes	Allowed, separate minutes	Allowed, separate minutes
CCM			—	Blocked for same practitioner	Allowed, separate minutes
PCM				—	Allowed, separate minutes
BHI					—

The engine enforces these at enrollment activation (hard block or warning per cell) and again at period close. A tenant may set any "Allowed" cell to "Blocked" through `payer_override`; no tenant can set a "Blocked" cell to "Allowed".

7. Billing engine

The billing engine is a pure function: given an enrollment, a period, the rule pack version, and the committed time entries, device days, interactions, consents and care-plan state inside that period, it returns the evidence per code and the blockers. It never writes to the tables it reads. Because it is pure, it is fully unit-testable and the same code runs live in the UI and at month-end close.

Periods

- A calendar-month period is created for every active enrollment on the 1st of the month in the tenant timezone, and on activation for mid-month starts.
- A rolling 30-day supply period for RPM and RTM starts on the first counted device day after setup or after the previous period ends. Periods never overlap for one enrollment.
- Period close runs at 02:00 tenant time on the 1st for the previous month, and daily for rolling periods that ended the day before. Close freezes `billing_evidence`, generates `billing_line` candidates, and locks the time entries inside the period (status stays `committed`, `billing_period_id` set, further edits require reopen).
- Reopen is a tenant-admin action with a reason; it is audited and re-runs the evidence calculation.
- Close and roll-up jobs are BullMQ repeatable jobs and are idempotent: running one twice for the same period gives the same result and creates no duplicates. This matters on Replit, where a process can restart at any time.

Minute counting (the rule that must never break)

1. A minute belongs to exactly one `time_entry`, and one `time_entry` belongs to exactly one program. Overlapping committed entries for the same user are impossible (database exclusion constraint).
2. Only `committed` entries count. Drafts and voided entries never count.
3. The entry's `user_credential_snapshot` decides which bucket it fills: practitioner minutes or clinical-staff minutes, as the rule pack defines. Entries from credentials the rule pack does not list are shown as "not countable" in the UI at the moment they are logged, not discovered at month end.
4. Only entries whose `activity_code` is in the rule pack's activity list count.
5. No rounding. 19 minutes is 19 minutes. Thresholds are compared exactly.
6. Add-on units are whole blocks beyond the base threshold: for 99458, units = floor((minutes − 20) / 20), capped by `max_units`.
7. When a patient is enrolled in two programs that both accept the same activity, the clinician picks the program when starting the timer. The default is the program whose worklist item opened the timer. The UI shows a running tally per program for the month so staff can see what is still short.

Device-day counting

- A device day is a calendar date in the patient's timezone with at least one valid observation from a source the rule pack counts (RPM: FDA-cleared device; RTM: the template's declared therapeutic streams, device or patient-reported as the code allows).
- Multiple readings on one day are one day. Readings received late but measured inside the period count for the measured date if they arrive before period close; after close they are recorded as late and excluded, with a visible note.
- Invalid readings (out of physiologic range, device error, duplicate) never count and are listed on the patient page with the reason.

Live interaction evidence

An `interaction` with `is_live_two_way = true` (phone or video with the patient or caregiver) inside the calendar month satisfies the management-code requirement for RPM and RTM. Secure messages and SMS never satisfy it. The UI shows "live interaction: none yet this month" on the patient header for RPM and RTM enrollments.

Evidence output (per code)

Field	Meaning
`met`	All requirements satisfied
`quantity`	Units the evidence supports
`minutes_counted`, `device_days_counted`, `live_interactions`	The raw inputs the decision used
`requirements`	Each requirement with its threshold, the observed value and pass or fail
`blockers`	Human-readable reasons the code is not met, each linking to the action that fixes it (for example "needs 4 more device days by 27 Nov", "consent not on file")

Screens and reports fed by the engine

- Patient header: a per-program progress strip for the current period (minutes, device days, live interaction, care-plan status).

- Program dashboard: patients at risk of missing a code this period, sorted by days left and shortfall. This is the single most useful operational view and must load in under one second for 5,000 enrollments.
- Month-end report: one row per enrollment per code with status candidate, approved, rejected; filters by location, billing provider, program, payer kind; export to CSV and XLSX; a PDF audit pack per patient on demand that lists every time entry, device day and interaction behind each approved line.
- Reconciliation test (CI): for a seeded month, total approved minutes across all lines equals total committed minutes in `time_entry` for that period, program by program. The build fails if they differ.
- A month generator (`pnpm seed:month`) creates a full synthetic month of readings, interactions and time for the 50 demo patients, so billing can be demonstrated and tested on Replit.

What the engine does not do

It does not pick diagnosis codes (they are copied from the enrollment's conditions and editable by billing staff), does not check payer coverage, does not compute dollars, and does not submit claims. A banner on the report states this in one sentence.

8. Care plans, templates, notes and time

A care plan is a structured document built from a template, signed by a named user, versioned on every change, and exportable as PDF. The clinical content is the customer's; the platform guarantees the required elements are present before signing and that every minute of work can be tied back to the plan.

Template structure (`careplan_template.sections`)

```
{
  "sections": [
    { "key": "problems", "title": "Problem list", "required": true,
      "fields": [{ "key": "conditions", "type": "condition_list", "min": 2 }]
},
    { "key": "goals", "title": "Measurable goals", "required": true,
      "fields": [{ "key": "goals", "type": "goal_list", "min": 1 }] },
    { "key": "interventions", "title": "Interventions and responsible party",
"required": true,
      "fields": [{ "key": "items", "type": "careplan_item_list", "min": 1 }]
},
```

```
    { "key": "medications", "title": "Medication management", "required":
true,
      "fields": [{ "key": "meds", "type": "medication_list" }, { "key":
"reconciled_at", "type": "date" }] },
    { "key": "symptoms", "title": "Expected outcome and prognosis",
"required": true,
      "fields": [{ "key": "text", "type": "rich_text", "min_chars": 40 }] },
    { "key": "community", "title": "Community and social services",
"required": false,
      "fields": [{ "key": "text", "type": "rich_text" }] },
    { "key": "coordination", "title": "Care coordination and other
practitioners", "required": true,
      "fields": [{ "key": "practitioners", "type": "practitioner_list" }] },
    { "key": "review", "title": "Review schedule", "required": true,
      "fields": [{ "key": "cadence_days", "type": "integer", "default": 30 }]
}
  ],
  "complexity_field": { "key": "complexity", "options": ["low",
"moderate_or_high"] }
}
```

Default templates shipped in `packages/rulepacks/careplans/`

Template key	Program	Required sections (beyond the common ones above)
`ccm_default`	CCM	Two or more chronic conditions, comprehensive plan covering all conditions, medication reconciliation date, complexity field
`pcm_default`	PCM	Exactly one qualifying condition, disease-specific goals and interventions, expected duration
`rpm_default`	RPM	Monitored parameters with thresholds, device assigned, reading schedule, escalation contact
`rtm_msk_default`	RTM (MSK)	Diagnosis, prescribed exercises with frequency, pain and function instruments and cadence, therapy plan-of-care reference

Template key	Program	Required sections (beyond the common ones above)
`rtm_respiratory_default`	RTM (respiratory)	Device or inhaler regimen, symptom instrument, action plan
`bhi_general_default`	BHI	Initial assessment, chosen validated scale and cadence, treatment plan, referral pathway
`bhi_cocm_default`	BHI (CoCM)	Everything in general plus behavioral care manager, psychiatric consultant, registry flag, consultant review cadence

Tenants copy a default and edit; the copy keeps a link to the default version so Mahalo can show a diff when the default changes.

Care plan lifecycle

1. Created in `draft` from the template when the enrollment is created; prefilled from EHR conditions and medications where an EHR link exists (every prefilled field is marked `source: ehr` until a user confirms it).
2. Signed by a user with the `sign_care_plan` permission. Signing validates required elements, stores a PDF, and sets `active`.
3. Any edit after signing creates a new version in `draft`; the active version stays active until the new one is signed. Diff view between versions.
4. Review tasks are generated on the template cadence; completing a review logs time and a `care_plan_review` row.
5. On discharge the plan is `closed` with a reason and a final PDF.

Care plan items drive the patient app

Each `care_plan_item` has an RRULE schedule and a window. A nightly job and an on-save hook materialize patient `task` rows for the next 14 days. Completing, skipping or missing a task writes an `adherence_event`. Readings and check-ins are items too, so one schedule engine serves medications, exercises, readings, instruments and education.

Notes

- Note templates per kind: monthly summary (auto-prefilled with the period's readings summary, exceptions handled, time totals, care-plan changes), encounter note, care-plan review, exception resolution.

- Rich text with structured header fields (patient, program, period, author, credential, time logged). Signed notes are frozen; amendments are new versions.
- The AI module may draft the monthly summary; the draft is shown in a distinct panel with an "Accept into note" button; nothing enters the note body without that click.

Time capture

- A timer control sits in the app header. Opening a patient from a worklist item starts a draft entry attributed to that item's program and activity; the user can change both before committing.
- Stopping the timer or navigating to another patient prompts commit, discard or keep running (max one running timer per user).
- Manual entries require start, end, activity and program, and are tagged `manual`. A tenant setting can require a reason for manual entries over 30 minutes.
- Daily "uncommitted time" reminder to the user; weekly report to the clinical admin of manual versus timer minutes per user.
- The AI module may suggest a time entry from a completed interaction (`ai_suggested`); it is a draft until accepted.

PDF export

`packages/pdf` holds HTML templates for the care plan, each note kind, the consent record and the per-patient billing audit pack. A worker renders them with Playwright to A4 and Letter, stores them through the storage adapter with the SHA-256 on the `document` row, and the UI offers download or "send to EHR" (which, in v1, means a signed download link the records clerk files in OmniMD; a later phase can attach it through an EHR document API).

9. Clinical rules, exceptions and worklists

Rules turn data into a short, owned list of exceptions. A rule has a kind, a definition, a severity, an owning role and a deduplication window. The worker evaluates rules on every new observation, check-in, adherence event and score, and in a nightly sweep for missing-data and time-budget rules.

Rule definition grammar (`rule.definition`)

Kind	Definition fields	Example
`threshold`	`loinc`, `op` (gt, gte, lt, lte, outside), `value` or `low`/`high`, `confirm_with_n_readings`, `within_hours`	Systolic ≥ 180 or diastolic ≥ 120 on any reading → critical
`trend`	`loinc`, `window_days`, `stat` (mean, slope, max), `op`, `value`, `min_points`	7-day mean systolic ≥ 150 with at least 4 points → high
`missing_data`	`stream` (device, epro, adherence), `days_without`, `only_if_enrolled_days_gte`	No readings for 3 days → medium, owner care manager
`adherence`	`item_kind`, `window_days`, `completion_rate_lt`, or `missed_consecutive_gte`	Exercise completion under 50% in 7 days → medium
`score`	`instrument_key`, `op`, `value`, or `band_in`, `change_from_prior_gte`, `item_flag`	PHQ-9 ≥ 20, or item 9 > 0 → critical, owner behavioral care manager, immediate notify
`device_days_at_risk`	`program_key`, `code`, `days_left_lte`, `shortfall_gte`	99454 needs 5 more days with 6 days left → low, owner care manager
`time_budget`	`program_key`, `code`, `days_left_lte`, `minutes_short_gte`	99490 needs 12 more minutes with 7 days left → low

Defaults per therapeutic template

- Cardiovascular (RPM for hypertension): critical BP, high trend, missing data 3 days, weight gain for heart failure (2 kg in 3 days) when a scale is assigned.
- Diabetes (RPM): glucose < 70 or > 300 critical, 7-day mean > 180 high, missing data 3 days.
- Musculoskeletal (RTM): pain NRS ≥ 8 high, pain up by 3 from baseline high, function score worsening high, exercise adherence under 50% medium, barrier reported `too_painful` or `side_effects` medium, missing check-in 5 days low.

- Behavioral health (BHI): PHQ-9 item 9 > 0 critical with immediate notification, PHQ-9 or GAD-7 severe high, no scale in 28 days medium.

These are starting values. Tenants may edit thresholds and owners; Mahalo reviews defaults with a clinician before release and records the reviewer on the template version (**VERIFY** each default with the Clinical Safety Officer before any tenant uses it).

Exception behavior

- One open exception per rule and patient. A new matching signal increments `signal_count` and updates `last_signal_at`; it does not create a second row.
- Severity escalates automatically if the rule defines `escalate_after_hours` and nobody acknowledged.
- Resolution requires a `resolution_code` (contacted_patient, adjusted_plan, referred, false_positive, device_issue, no_action_needed) and may link a note. Dismissing a critical exception requires a reason.
- Every state change is audited with the user and time; the time spent is captured through the timer that opens with the exception.

Worklists (server-side views, paginated, virtualized)

Worklist	Who	Sort	Row shows
My exceptions	Any clinical user	Severity, then age	Patient, program, rule, latest value, age, one-click actions (call, message, resolve)
Team queue	Role	Severity, then age	Same, unassigned items first
Due today	Care manager	Due time	Reviews, follow-ups, scheduled calls, consent renewals
At risk this period	Care manager, billing	Days left, then shortfall	Patient, program, code, what is missing, days left
Unsigned	Billing provider	Age	Draft care plans and notes awaiting signature

Worklist	Who	Sort	Row shows
Device issues	Front desk, care manager	Age	Unreturned devices, no first reading after 7 days, low battery, setup incomplete
CoCM registry	Behavioral care manager, psychiatric consultant	Days since consultant review	Patient, last PHQ-9 and GAD-7, trend, last contact, consultant review due

All worklists filter by location (multi-select, default to the user's primary locations), program and assignee, and remember the last filter per user.

Notifications

A rule's `notify` block names channels (in_app, push, sms, email) and a quiet period. Critical exceptions may page a named on-call role through SMS; the message says only "A patient needs attention in Mahalo Care" with a deep link. Patients receive reminders and nudges, never clinical alerts; a patient who reports a critical value sees instructions configured by the tenant (for example, call 911 for chest pain) and the care team is notified in parallel. On Replit the SMS and email drivers write to the log, so notification flows are testable without sending anything.

10. Web applications: workspace, admin, patient PWA

Three web surfaces share one API and one component library. The workspace has a left navigation of seven items and no more; everything else is reached from a patient page or a command palette (⌘K) that searches patients, worklists, settings and actions.

Clinician workspace (`apps/web`) — routes

Route	Purpose	Key components
`/home`	Today: my exceptions, due today, at-risk count, uncommitted time	Three stacked worklist cards, timer in header
`/worklists/:key`	Full worklists from section 9	Virtualized table, saved filters, bulk assign

Route	Purpose	Key components
`/patients`	Search and panel view	Trigram search, filters by location, program, status, enrolled since
`/patients/:id`	Patient page, the center of the product	Header (identity, home location, programs with period progress strips, live-interaction status, timer), tabs below
`/patients/:id/timeline`	Everything in time order: readings, check-ins, adherence, messages, calls, notes, exceptions, time	Infinite scroll, filter chips by kind
`/patients/:id/readings`	Charts per parameter with thresholds drawn, table of raw values, invalid readings listed	One chart per LOINC, 7/30/90-day ranges
`/patients/:id/care-plan`	Active plan, versions, review, sign, export PDF	Section accordion, required-element checklist, diff view
`/patients/:id/programs`	Enrollments: status, consents, devices, billing provider, discharge	Activation checklist with concurrency warnings
`/patients/:id/notes`	Notes list and editor	Template picker, AI draft panel, sign
`/patients/:id/time`	Time entries for this patient, by program and period	Table, add manual, void with reason
`/patients/:id/messages`	Secure thread and call log; start a call or video	Thread view, interaction logging prompt after a call
`/patients/:id/ehr`	Read-only EHR context from the cache	Conditions, meds, vitals, results, last synced
`/programs`	Program dashboards: enrolled, active, at risk, period progress	Cards per program, at-risk table

Route	Purpose	Key components
`/billing`	Month-end: evidence, candidates, approve, reject, export, audit pack	Period picker, grouped by provider and location, bulk approve with confirm
`/reports`	Operational reports: enrollment funnel, device fleet, staff time, exception response times	Saved report definitions, CSV export
`/admin/*`	Tenant admin (see below)	

The patient page header is the most important design element. It must show, without scrolling: name and photo, DOB and MRN, home location, each active program as a chip with a progress strip (minutes logged / threshold, device days / 16, live interaction yes or no, care plan active or draft), open exceptions count by severity, next due item, and the timer. Everything on the header is a link.

Admin (`/admin`)

- Locations: add, edit, timezone, group NPI, place of service.
- Users and roles: invite, credential, NPI, roles per location, MFA status, deactivate.
- Programs: create from a rule pack and a template, name it, set default billing provider and consent text, activate.
- Templates: therapeutic templates and care-plan templates, copy from default, edit thresholds and content, version and publish with a reviewer name.
- Rules: list by program, edit thresholds and owners, test a rule against a patient's history before publishing.
- Devices: catalog enablement, fleet inventory, vendor credentials, webhook health.
- EHR: connection setup, test, sync status, field mapping for custom adapters.
- Branding: logo, colors within the design-system constraints, patient app name, support phone, consent texts.
- Payer overrides: tighten rules per payer.
- Configuration: export and import of programs, templates, rules and care-plan templates as JSON (the same function as `pnpm config:export` and `config:import`), so a tenant set up on Replit can be moved to GCP.
- Audit: search the audit log by patient, user, action and date; export.

Patient PWA (`apps/patient`)

The PWA is for patients who will not install the app and for the first minutes after an invite link is tapped. It has the same screens as the patient mobile app in section 10a, except Bluetooth pairing and HealthKit or Health Connect, which need the native app. Cellular-device patients can use only the PWA and never need Bluetooth.

Cross-cutting UI rules

- Every list has a saved default filter per user and a URL that encodes the filter so links can be shared.
- Every destructive or clinical action (sign, void, discharge, dismiss critical) has a confirm dialog that states the consequence in one sentence.
- Optimistic updates for low-risk actions (assign, acknowledge); server-confirmed for clinical ones (sign, commit time).
- Offline: the workspace shows a banner and queues nothing; the patient apps queue check-ins and sync when back online.
- Performance budgets are tested in CI with Lighthouse on the patient PWA and Playwright timing on the three heaviest workspace routes.

10a. React Native mobile apps: patient app and clinician companion (v1)

Two React Native apps ship in v1 from one Expo codebase, one JavaScript bundle each for iOS and Android, no Swift or Kotlin code written by the team: the patient app (the main way patients take readings, do exercises and check in) and a clinician companion app (worklists, calls, timer and quick notes on the move). Both share the API and `packages/contracts` ; no logic is duplicated from the web apps.

Stack

Concern	Choice
Framework	Expo SDK (pinned), React Native, TypeScript strict, expo-router for file-based navigation
Data	TanStack Query with persisted cache, Zustand for UI state, offline queue in SQLite (expo-sqlite)

Concern	Choice
Styling	NativeWind with the same design tokens as `packages/ui`; a `packages/ui-native` library mirrors the web components
Auth and secrets	Better Auth mobile client, tokens in expo-secure-store, biometric unlock (expo-local-authentication)
Devices	react-native-ble-plx for Bluetooth; react-native-health (HealthKit) and Health Connect on Android; one adapter per device model in `packages/device-ble`
Push	expo-notifications over FCM and APNs; payloads carry no clinical content
Video and calls	Daily React Native SDK; phone calls through the native dialer with an in-app log prompt
Media	expo-av for exercise videos, downloaded for offline use per care plan
Monitoring	Sentry React Native with PHI scrubbing; OpenTelemetry traces to the API
Build and release	EAS Build, EAS Submit, EAS Update for JavaScript-only fixes; native-module changes go through the stores. Only Expo modules and config plugins, so the team never maintains Xcode or Android Studio projects by hand
Testing	Jest and React Native Testing Library; Maestro end-to-end flows on a device farm in CI
Where it runs while building	Metro dev server in the Repl and Expo Go for UI work; EAS development builds on real devices for Bluetooth and health data

Patient app — screens

Screen	What it does
Onboarding	Opens from an SMS or email invite with a one-time code; sets PIN and biometrics; shows the program consent text in the patient's language; captures device-sharing and communication consents
Home	Today's tasks as cards: take reading, exercise, check-in, medication, education; a device status strip (connected, last reading, battery); one message badge; nothing else
Take reading	For cellular devices: shows the last value and when it arrived. For Bluetooth: pair, read, confirm; the value is signed and sent as `source = device`. Manual entry only where the program allows it, stored as `patient_reported`
Exercise	Video or step list, timer, done or "couldn't, because…" with barrier codes; videos cached offline
Check-in	Validated instruments (PHQ-9, GAD-7, pain, function) one question per screen, large buttons; crisis text configured by the tenant appears when a rule says so, with the care team notified in parallel
Medication	Taken, skipped with reason, schedule view, refill reminder text
Progress	This week versus last, plain-language summaries approved by the tenant; no gamification pressure
Messages and video	Secure thread, join video, request a call back
My plan	Readable care plan, goals, care team, support phone
Devices	Pair, troubleshoot, disconnect; HealthKit and Health Connect consent with a clear list of what is read

Screen	What it does
Settings	Language, reminders and quiet hours, consent choices, caregiver access, delete my app data

Patient app rules: Atkinson Hyperlegible, minimum 17 pt body, 48 pt touch targets, Dynamic Type, VoiceOver and TalkBack labels on every control, works offline for check-ins and exercise logs and syncs when back online, never shows a clinical alert.

Clinician companion app — screens

Screen	What it does
Sign in	Tenant pick, SSO or password plus MFA, biometric unlock after first login
Today	My exceptions and due-today items, same sort as the web worklists
Exception	Patient header, the triggering value with trend, one-tap call (logs an interaction and starts the timer), message, acknowledge, resolve with code
Patient	Header with program progress strips, readings charts, recent timeline, care plan read-only, notes read-only
Timer	Running timer with program and activity, commit or discard
Quick note	Short note with dictation through the device keyboard; saved as draft for signing on the web
Messages and video	Secure threads, start or join video

The companion app does not do admin, billing approval, care-plan editing or note signing; those stay on the web where the full context is.

Device data path on mobile

1. The app pairs a catalog device over Bluetooth and stores the pairing per patient.
2. On read, the app builds a normalized reading, signs it with the device session key, and posts `POST /observations/batch`; failed posts are queued and retried.

3. The API writes `observation` with `source = device` and the catalog id, so FDA-cleared BLE devices count toward RPM device days exactly as cellular ones do.
4. HealthKit and Health Connect data is read only with consent, stored as `source = wearable`, and never counts toward device days.

Security on mobile

- Tokens in the secure enclave or keystore only; sessions revoked server-side on sign-out and on user deactivation (remote wipe of cached data).
- Biometric or PIN lock after 2 minutes in the background on the clinician app, 5 minutes on the patient app.
- Screens with PHI block screenshots and recent-apps previews (Android FLAG_SECURE, iOS blur on background).
- Certificate pinning to the API in production builds (not in Replit-era builds that point at a Repl URL); jailbreak and root detection shows a warning and disables the clinician app.
- No PHI in push payloads, crash reports, logs or analytics; analytics are event counts only.
- Compatible with MDM for clinician devices; no requirement for it.

API base URL by build

The API address is set per build profile in `eas.json`: a `replit` profile points at the Repl URL for development builds with synthetic data; `staging` and `production` profiles point at the GCP HTTPS hostnames. A store build can never be created from the `replit` profile, and the PHI guard also rejects a `replit` build talking to a production API.

White label and store listings

- One patient app listing ("Mahalo Care") with tenant theming by invite code in v1; a dedicated listing for a tenant is a configuration of the same build and a separate submission.
- Store review notes, privacy nutrition labels and Health Connect declarations are kept in `apps/patient-mobile/store/` and updated on every release.

Release process

- Every PR builds an internal distribution for both apps; QA installs from a link.
- `rc-*` tags go to TestFlight and the Play internal track for staging; `v*` tags are submitted to the stores after the web release is live.

- JavaScript-only fixes may ship through EAS Update within store rules; anything that changes a native module or permission waits for a store release.
- Minimum OS: iOS 16, Android 10; Bluetooth features require Android 12+.

11. Integrations: devices, EHR, messaging, video, PDF

Every external system sits behind an adapter interface in `packages/adapters` (section 4). The core never imports a vendor SDK directly, so a vendor can be added or replaced by one new file and one catalog row. Each integration has a Replit-safe driver (simulator, fixture, console or sandbox) so it can be built and tested with synthetic data before any live account exists.

Devices

Path	Phase	How it works
Cellular devices via vendor cloud	v1	Vendor pushes readings to `POST /webhooks/devices/:vendor`; adapter verifies the signature, maps to LOINC and units, writes `ingest_inbox` then `observation`. Candidates: Tenovi, Smart Meter, Withings, Omron (**VERIFY** vendor choice and BAA with the first customer).
Vendor pull	v1	For vendors without webhooks, a scheduled pull every 15 minutes with a per-vendor cursor.
Bluetooth via the React Native app	v1	The app pairs BLE devices, reads, signs the payload with the device session, posts `POST /observations/batch`.
HealthKit and Health Connect	v1	Consented reads of steps, sleep, heart rate, workouts; stored as `source = wearable`, never counted toward RPM device days.
Manual entry	v1	Allowed only where the program permits (RTM self-report, BHI scales); stored as `patient_reported`.

Path	Phase	How it works
Simulator	Build and demo	`tools/device-sim` posts signed fake webhooks for any catalog device with realistic patterns (normal, rising blood pressure, missed days, invalid readings). It is how readings, device days and exceptions are demonstrated and tested on Replit.

Adapter interface: `verify(request) → boolean`, `parse(payload) → NormalizedReading[]`, `health() → status`. A normalized reading carries vendor id, device serial, LOINC, value, unit, measured-at with offset, battery and signal where available. Webhook URLs on Replit are the Repl's public URL; for stable vendor sandbox registration use a Replit deployment URL (synthetic data only).

EHR

Path	Phase	How it works
FHIR R4 read (Epic, Oracle Health, athenahealth, NextGen, eClinicalWorks, MEDITECH, Allscripts)	v1	SMART backend services or client-credentials auth per connection; reads Patient, Condition, MedicationRequest, Observation (vitals, labs), Encounter; stored in `ehr_resource_cache`; nightly refresh plus on-demand refresh from the patient page. Built and tested against recorded fixtures and public FHIR sandboxes.
Custom adapter: OmniMD	v1 (scoped)	Same adapter interface over OmniMD's API. Build order: patient demographics lookup by MRN, then conditions and medications, then document attach if their API offers it. Scope standard versus custom with the customer's API owner before committing a date; never promise plug-and-play.
Patient matching	v1	Match on MRN or EHR patient id first; otherwise name plus DOB with a human confirm step. Unmatched patients are flagged, never auto-linked.

Path	Phase	How it works
Write-back	Later phase	Document attach (PDF) where the vendor supports it; discrete data write-back only after separate engineering verification and customer sign-off.

EHR data is read-only context in v1 and is labelled as such on every screen that shows it.

Messaging and notifications

- Secure messaging is in-app only. SMS and email carry no clinical content; they say a task or message is waiting and deep-link to the app.
- SMS: Twilio with a BAA, 10DLC registered, opt-out handled, delivery receipts stored on `notification`.
- Email: Postmark or SendGrid with a BAA, transactional only.
- Push: web push for the PWA where the browser allows, FCM and APNs for the apps.
- Every outbound message is a `notification` row with provider id and status; a bounce or failure creates a staff task.

Video

Daily.co (or Twilio Video) with a BAA. A video call is started from the patient page or a message thread; the app creates a `video_session`, an `interaction` with `is_live_two_way = true`, and starts the timer. No recordings in v1. The patient joins from the app or from an SMS link with a one-time token valid for 30 minutes.

PDF and documents

Templates in `packages/pdf` are React components rendered to static HTML, then to PDF by Playwright in the worker. Documents are stored through the storage adapter under `tenant/<id>/patient/<id>/<kind>/<document id>.pdf`, served by 15-minute signed URLs, and every download is audited. The care plan PDF carries a footer with version, signer, signature time and SHA-256.

Exports and APIs for the customer

- CSV and XLSX exports from billing and reports.
- A read-only tenant API (OAuth client credentials) exposing patients, enrollments, billing evidence and lines, for a customer's own BI. Rate-limited, audited, and documented from the generated OpenAPI.
- Webhooks out (period closed, line approved, critical exception) are a later phase.

12. Mahalo Intelligence (governed AI)

AI in this product drafts, summarizes and ranks; it never decides. Every AI output is an `ai_draft` row with a status, and nothing reaches a clinical field, a note, a time entry or a billing line without a named user accepting it. That rule is enforced in the database (the AI service account has no write permission on clinical tables) and in the API.

Capabilities in v1

Capability	Input	Output	Where it appears
Patient summary	Last 30 days of readings, check-ins, exceptions, notes, care-plan changes	5 to 8 plain sentences with the numbers cited	Patient page header "Summary" drawer, regenerated on demand
Monthly note draft	The period's data plus the note template	Filled template sections, marked draft	Note editor side panel, "Accept into note"
Exception prioritization	Open exceptions for the user's queue	A suggested order with one-line reasons	Worklist "Suggested order" toggle; the default sort is always the rule-based one
Message draft	Thread context and the tenant's tone guide	A reply draft at a 6th-grade reading level	Message composer, "Use draft"
Time suggestion	A completed call or video session	A draft time entry with activity and program	Timer area, "Commit" or "Edit"
Barrier clustering (later phase)	Adherence barriers across a program	Themes with counts	Program dashboard

Provider and data handling

- Provider interface in `packages/adapters/ai` with these drivers: `anthropic` (direct Anthropic API, allowed only when `DATA_CLASS=synthetic`, which is the Replit and dev setting), `vertex` (Claude on Vertex AI under the Google BAA, region-pinned, the production default), `openai` (kept for parity with the existing Mahalo app; usable only where the tenant has a BAA with OpenAI), and `mock` (fixed outputs for tests).

- The PHI guard (section 0) refuses to start with `DATA_CLASS=phi` unless `AI_DRIVER` is a driver covered by a BAA. This is what stops a Replit-style configuration from ever touching real patient data.
- Prompts are versioned files in `packages/prompts` with a changelog; `ai_draft.prompt_version` records which one produced the output.
- The minimum necessary data is sent: identifiers are replaced with tokens before the call and restored after; no free-text fields that could carry third-party names unless the capability needs them.
- No training on customer data; zero-retention settings on the provider where available.
- `ai_call_log` records tokens, latency and prompt hash, never the content.

Guardrails

- Every draft shows a header: "AI draft — review before use" and the data range it used.
- Numeric claims in a summary must appear in the source data; a post-check compares every number in the output with the inputs and flags mismatches in red. A draft with a mismatch cannot be accepted until edited.
- The AI never produces a diagnosis, a medication change, a billing eligibility statement or an emergency instruction. Prompts forbid it and an output classifier blocks those phrases; blocked drafts are logged and shown as "not available".
- Acceptance, edit and rejection rates per capability are reported monthly to Mahalo product; a capability with under 50% acceptance is reviewed.

Evaluation

`packages/prompts/evals` holds a fixed set of synthetic patient months with expected summaries. CI runs the eval on every prompt change and fails on regressions in the numeric-fidelity check or the forbidden-phrase check. The eval runs against the `anthropic` driver in CI with a CI-only key, and against `mock` for fast local runs.

13. Security, privacy and compliance

The platform is built to HIPAA Security Rule controls from the first commit, aligned with Mahalo's existing ISO 27001, SOC 2 Type II and ISO 13485 programs. Compliance is a property of the code and infrastructure, not a document written afterwards. Building on Replit is safe only because no real patient data is ever there; the controls below make that a technical fact, not a promise.

Replit build-phase controls

- Synthetic data only. The PHI guard (`DATA_CLASS`) stops the app from starting with real data outside production, and the seed and simulator tools generate every record.
- Private Repl, named members, two-factor on every account, no shared logins. Replit Secrets hold test credentials only; production secrets exist only in GCP Secret Manager.
- No production credentials, production database URLs or live vendor keys are ever pasted into Replit, GitHub issues or Linear.
- Any new team member or contractor on the Repl signs the same confidentiality terms as for production.
- A weekly review of Repl members and Secrets, recorded as ISMS evidence.

Identity and access

- MFA (TOTP or passkey) is mandatory for every workspace user; the tenant may add OIDC single sign-on. Sessions expire after 12 hours of inactivity and 24 hours absolute; a 15-minute idle lock in the workspace.
- Role-based access with location scope. Permissions are checked in the API on every route through one `authorize(user, action, resource)` function; the UI only hides what the API forbids.
- Cross-site access: a user sees patients at locations in their role scope. Opening a patient outside that scope is a break-glass action with a reason, time-limited to 4 hours, audited and reported to the clinical admin weekly.
- Patients authenticate with phone or email one-time codes plus an optional PIN; caregivers get a separate login with patient-granted scope.
- Platform admins (Mahalo support) use a separate identity provider, hardware keys, and see tenant data only through a support session the tenant admin approves; every such session is logged and the tenant can view the log.

Data protection

- Encryption in transit everywhere (TLS 1.2+); at rest with CMEK on Cloud SQL, Cloud Storage and backups; application-level encryption for a short list of fields (`patient.phone`, `patient.email`, `coverage.member_id`) through the crypto adapter (`gcp-kms` envelope keys in production).
- Logs never contain PHI: a pino redaction list plus a CI test that greps log fixtures for names, phones and MRNs. Request bodies are never logged.
- Backups daily with 35-day retention; quarterly restore test documented in the runbook.

- Data residency is a tenant attribute; US tenants run in US regions only.
- Retention: clinical records and audit logs 7 years minimum (tenant-configurable upward); `ingest_inbox` 2 years; notifications 1 year.

Audit

- `audit_log` is append-only (INSERT-only role, trigger blocks UPDATE and DELETE), partitioned by month, exported nightly to a locked Cloud Storage bucket with object retention lock.
- Logged: every read of a patient chart, every write to clinical tables, every sign, void, reopen, export, download, break-glass, role change and support session.
- Tenant admins can search and export their audit log from `/admin/audit`.

Application security

- Zod validation on every input; Drizzle parameterized queries only; no raw SQL string building.
- Content Security Policy, strict CORS, HttpOnly SameSite cookies, CSRF tokens on state-changing routes.
- Rate limits on auth, webhooks and patient endpoints (Cloud Armor plus in-app).
- Dependency scanning (GitHub Dependabot and `pnpm audit`) and container scanning in Artifact Registry; critical findings block release.
- Secrets only in Replit Secrets (test values) or Secret Manager; a pre-commit hook and CI check reject secrets in code.
- Annual third-party penetration test; findings tracked in Linear with due dates. A first test runs against the staging environment on GCP before the first real tenant.

Privacy and consent

- Consent texts are versioned per tenant and program; the patient sees the version they agreed to.
- Patients can disconnect devices, change communication preferences and revoke caregiver access from the app at any time.
- A data export per patient (JSON plus PDFs) is available to the tenant for access requests.

Compliance artifacts generated from the system

- Access review report (users, roles, last login) monthly.
- Audit log integrity report (row counts and hash chain) monthly.

- Device and integration health report weekly.
- These feed Mahalo's ISMS evidence collection (Pono) rather than being written by hand.

Regulatory position to state consistently

Mahalo Care is a care-management and monitoring platform. It is not a medical device in v1; clinical thresholds are configured and owned by the customer's clinicians and no feature diagnoses or recommends treatment. If a future feature crosses into software as a medical device, it goes through the ISO 13485 design-control process before release.

14. Multi-tenancy, multi-site and white label

One deployment serves many tenants; one tenant has many locations; one patient has one home location and can be seen at any location by staff whose role covers it. This is the model to confirm live with the customer (their "master location" question), and the data model already supports every variant they are likely to describe.

Tenant isolation

- Shared database, row-level security on `tenant_id`, one API, one deploy. A tenant can be moved to a dedicated Cloud SQL instance later without code change because every query already carries `tenant_id`.
- Tenant resolution is a setting, `TENANT_RESOLUTION`. `subdomain` (production): `groth.care.mahalo.health` selects the tenant. `membership` (Replit and dev): Replit gives no wildcard subdomains, so the tenant comes from the user's membership after login, with a dev-only `?tenant=` switch for seeded demo tenants. Patients resolve by the invite link or the app build.
- Per-tenant settings: timezone, region, branding, consent texts, default programs, payer overrides, feature flags, data retention.
- Per-tenant rate limits and storage quotas with alerts, so one tenant cannot degrade another.

Locations and the home-location model

- `patient.home_location_id` is the billing and reporting home. It appears on every billing line and defaults from the enrolling location.
- Changing it is an audited action with a reason; the change applies to future periods only, and the current open period keeps the old location unless billing staff explicitly reassign it.

- Staff access is by role scope (set of locations or all). A patient seen at a non-home office is simply opened by a staff member whose scope includes the patient's home location; no record moves.
- Reports and worklists filter by home location, by the location of the acting user, or both.
- Each location carries its own timezone (relevant for multi-state groups), place-of-service code and group NPI.

Alternative models the customer may describe, and how they map

What they might say	How it maps
"One central billing entity for all offices"	One location flagged `is_billing_home` for all enrollments, other locations are service sites; billing lines carry both
"Each office bills its own patients"	Default behavior: home location equals enrolling location
"Patients move between offices and whoever sees them should see everything"	Role scope = all locations for clinical roles; home location unchanged
"A central monitoring team serves all offices"	A `care_manager` role with scope = all; worklists grouped by home location

White label

- Branding per tenant: logo, primary color (must pass contrast checks against the design-system canvas), patient app display name, support phone and email, consent and welcome texts, email sender name.
- Patient surfaces: one PWA with runtime theming by tenant; the React Native app ships as one store listing with tenant selection by invite code, with the option of a dedicated listing for large tenants.
- Mahalo's name appears only in a "Powered by Mahalo Health" line in the footer unless the contract removes it.

Reseller and sub-tenant (later phase)

A `parent_tenant_id` on `tenant` lets a partner organization manage several practice tenants with roll-up reporting and no cross-tenant data access beyond aggregates.

15. SDLC: repo, GitHub, Linear, Replit workflow, CI/CD

The SDLC follows the published Mahalo SDLC (mahalo.health/about/mahalo-sdlc): every change starts as a Linear issue, lands through a reviewed pull request with passing checks, and reaches production by promoting a tested image. Claude Code works inside this loop and never outside it. Replit is where the code runs during the build; GitHub is where it lives.

Repository (`github.com/Mahalo-Digital-ventures-Inc/mahalo-care`)

```
mahalo-care/
  apps/
    api/              Fastify API (modules per section 3)
    web/              Clinician and admin workspace
    patient/          Patient PWA
    worker/           BullMQ workers, scheduler, PDF rendering
    patient-mobile/   React Native patient app (Expo)
    clinician-mobile/ React Native clinician companion (Expo)
  packages/
    db/               Drizzle schema, migrations, seed
    config/           Environment parsing and the PHI guard
    adapters/         Driver interfaces plus local and production drivers
    contracts/        Zod schemas, generated OpenAPI and TS client
    rulepacks/        Program rule packs, care-plan templates, therapeutic
templates, schema
    ui/               Web component library and Storybook
    ui-native/        React Native component library
    mobile-core/      Shared mobile logic (auth, offline queue, API client)
    device-ble/       One Bluetooth adapter per device model
    pdf/              PDF templates
    prompts/          AI prompt versions and evals
  tools/
    device-sim/       Signed fake device webhooks for demos and tests
    config-export/    Export and import of tenant configuration
    seed/             Synthetic patients, month generator, reference-data
seed
    tenant-admin/     Create a tenant and its first admin (platform admin
only)
  docker/
    api.Dockerfile, worker.Dockerfile, web.Dockerfile, nginx.conf
  infra/
    terraform/        Modules: network, sql, redis, run, storage, kms, lb,
monitoring; envs: dev, stg, prod
```

```
  docs/
    adr/              Architecture decision records (one per decision,
numbered)
    runbooks/         Deploy, rollback, restore, incident, period-close
    spec/             This document exported as Markdown, kept in sync
    gcp-deployment-guide.md   Section 18 of this document, for DevOps
  .github/
    workflows/        ci.yml, images.yml, nightly.yml, deploy-dev.yml,
deploy-stg.yml, deploy-prod.yml
    PULL_REQUEST_TEMPLATE.md, CODEOWNERS, dependabot.yml
  .replit, replit.nix, replit.md     Replit configuration (section 4)
  CLAUDE.md           Operating instructions for Claude Code (section 17)
  package.json, pnpm-workspace.yaml, turbo.json, .tool-versions
```

Working in Replit (the daily loop)

1. Open the Repl, fetch from GitHub, and check out the branch for the Linear issue. Never commit directly to `main` from the Repl.
2. Run Claude Code in the Replit shell (or locally), make the change, and run `pnpm dev` to see it. Use the seeded demo tenant and the device simulator for data.
3. Run `pnpm lint typecheck test` and the API integration tests (`TEST_DB_MODE=url`) in the Repl. Push the branch; GitHub Actions runs the full set.
4. Fix what CI finds, open the pull request as a draft, then a human marks it ready.
5. After merge, pull `main` into the Repl. Configuration files (`.replit`, `replit.nix`, `replit.md`) are in git; if the Repl drifts, recreate it from git rather than editing by hand.
6. Secrets are set once per Repl in Replit Secrets from `docs/runbooks/replit-secrets.md` (names and purpose only, never values). `.env` files are git-ignored.

Branching and commits

- Trunk-based: `main` is always deployable. Feature branches are named `<linear-key>-<slug>` (for example `CARE-142-time-entry-exclusion`). No long-lived branches.
- Conventional commits (`feat:`, `fix:`, `chore:`, `db:`, `infra:`, `docs:`) with the Linear key in the footer so Linear links commits and PRs automatically.
- Squash merge; the PR title becomes the commit message. Signed commits required (GitHub verified).
- Releases are tags: `rc-YYYY.MM.DD-n` for staging, `v1.x.y` for production, cut by the release workflow, never by hand.

Pull request rules

- PR template asks for: Linear issue, what changed, how it was tested, migration yes or no, PHI impact yes or no, screenshots for UI, and "Replit-only assumption introduced: yes or no" (must be no).
- Required checks: lint, typecheck, unit, API integration (Testcontainers Postgres and Redis), migration up-and-down test, row-level-security isolation test, contract check (OpenAPI diff must be intentional), build of every app, **Docker build of api, worker and web with a container smoke test** (starts the image, waits for `/healthz`, hits `/readyz`), Storybook tests, Playwright smoke, secret scan, dependency audit, and the PHI-guard test (the app must refuse to start with `DATA_CLASS=phi` and a non-production driver).
- CODEOWNERS: `packages/db` and `packages/rulepacks` require a second reviewer; `packages/rulepacks` changes require a clinical-ops reviewer in addition; `docker/` and `infra/` require the DevOps lead.
- A PR that touches `billing` must add or update a reconciliation test case.
- Claude Code opens PRs as draft; a human marks ready for review. Claude Code may address review comments on its own PRs.

Linear setup

- Team `CARE`. Projects: one per milestone in section 16 (M0 to M7). Cycles: two weeks.
- Labels: `area:api`, `area:web`, `area:patient`, `area:mobile`, `area:worker`, `area:db`, `area:infra`, `area:rulepack`, `area:replit`, `area:gcp`, `type:feature`, `type:bug`, `type:chore`, `type:spike`, `phi`, `verify-cms`, `blocked-customer`.
- Issue template: user story, acceptance criteria as checkboxes, test notes, out of scope.
- Workflow states: Backlog, Ready, In Progress, In Review, QA, Done, Cancelled. Until staging exists on GCP, QA means verified on the Replit demo build; after that, on staging. An issue moves to Done when a human verifies it.
- Every issue belongs to a milestone; nothing is worked that is not in a cycle.

CI/CD (GitHub Actions, Workload Identity Federation to GCP)

Workflow	Trigger	What it does
`ci.yml`	Every PR and push to `main`	Install, lint, typecheck, unit, integration against Testcontainers Postgres and Redis, migration up and down, RLS isolation test, PHI-guard test, contract diff, build all apps, Storybook tests, Playwright smoke, upload coverage
`images.yml`	Every PR and push to `main`	Builds the api, worker and web Docker images, runs the container smoke test, scans them, and on `main` pushes them to Artifact Registry tagged with the commit SHA (the push step is skipped until the GCP project exists)
`mobile.yml`	PR touching mobile apps or shared mobile packages	Type check, Jest, EAS internal build, Maestro flows
`deploy-dev.yml`	Push to `main`, enabled when repository variable `GCP_ENABLED` is `true`	Promotes the image to the GCP dev project, runs the migration job, deploys, runs the full Playwright suite
`deploy-stg.yml`	Tag `rc-*`	Promotes the dev image by digest, migrates, deploys to staging, runs the full suite plus the billing reconciliation seed month
`deploy-prod.yml`	Tag `v*` with GitHub Environment approval (two approvers)	Promotes the staging image by digest, backup check, migrations with expand-and-contract pattern, canary 10% for 15 minutes with error-rate gate, then 100%; automatic rollback on gate failure
`nightly.yml`	03:00 UTC	Dependency audit, container rescan, AI prompt evals, restore test on dev from last backup (once dev exists), performance budget run

Database migrations

- Drizzle migrations are generated and committed; a migration must be backward compatible with the previous release (expand first, contract in a later release). CI runs up, then down, then up on a seeded database.
- Migrations run as a separate command (`pnpm db:migrate`) using `DATABASE_MIGRATE_URL`, never at API start-up. On GCP this is a Cloud Run job that runs before each deploy.
- Destructive migrations require an ADR and a tenant-admin notice.

Quality gates for release

- Coverage: 85% on `billing`, `time`, `rules`, `rulepacks`; 70% elsewhere.
- Zero open critical or high security findings.
- All Playwright flows green on the target environment: enroll, consent, assign device, receive reading, exception to resolution, log time, sign care plan, sign note, close period, approve line, export PDF.
- Runbooks updated if any operational step changed.

Observability and on-call

- On Replit: JSON logs in the console and a `/healthz` and `/readyz` endpoint per service; enough to debug, not a substitute for production monitoring.
- On GCP: dashboards for request latency and error rate per service, queue depth and job failures, webhook success rate per vendor, period-close duration, AI acceptance rate.
- Alerts to a Mahalo on-call rotation: API 5xx over 1% for 5 minutes, queue backlog over 1,000, webhook failure rate over 5%, period close not finished by 04:00 tenant time, Cloud SQL CPU over 80% for 15 minutes.
- Incident runbook with severity levels and customer notification templates; post-incident review as an ADR-style document in `docs/runbooks/incidents/`.

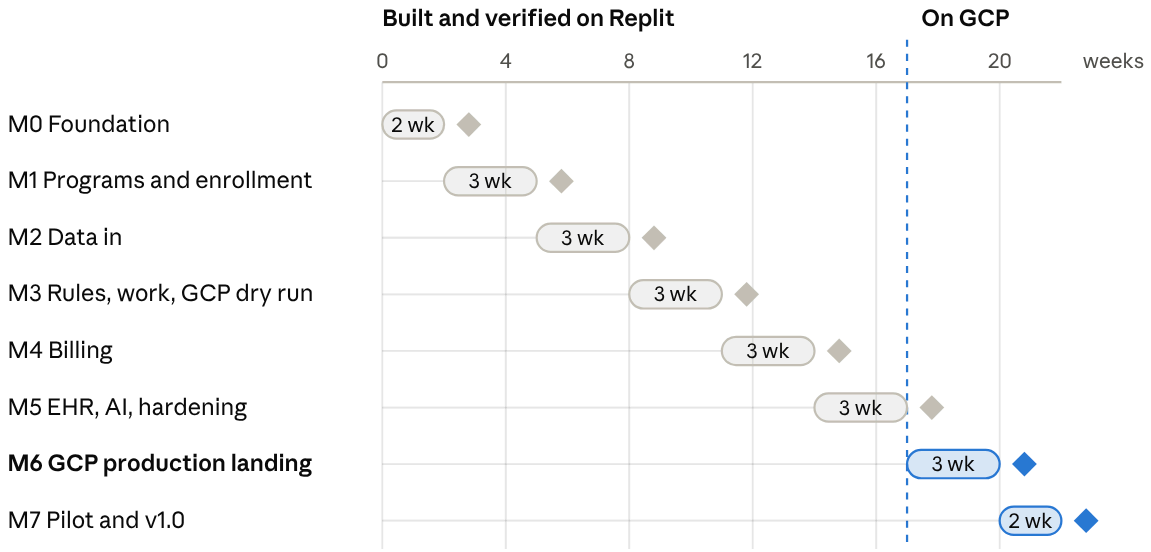
16. Delivery plan: milestones, epics, acceptance

Eight milestones, each a Linear project, each ending in a gate a human verifies. M0 to M5 are built and verified on Replit; M3 includes a short GCP dry run so surprises show up early; M6 lands production on GCP; M7 is the pilot. Durations are planning placeholders to

re-plan after M0; the order and the gates are fixed. Claude Code creates the Linear projects, milestones and the first issues from the epic lists below as its first task.

Production lands on GCP in week 17 of about 22; M0 to M5 run on Replit

Planning estimate in weeks, re-planned after M0. A diamond is a gate a person verifies.



delivery roadmap · 8 milestones, about 22 weeks, gates marked

The dashed line is the switch from Replit to GCP; each diamond is a gate in the table below.

Milestone	Estimate	Epics (Linear issues are children of these)	Gate to pass
M0 Foundation	2 weeks	Monorepo and tooling; Replit files and secrets runbook; `packages/config` with the PHI guard; adapter interfaces with local drivers; Dockerfiles and the CI image build with container smoke test; Drizzle schema for tenancy, identity, patients; Better Auth with MFA; RLS with the `app_rw` role and isolation test; audit log; design tokens and first 20 components; Expo workspace and `eas.json` profiles; seed data; CLAUDE.md and replit.md; spikes for Chromium PDF, Redis, ports and database extensions on Replit	A fresh Repl from git runs `pnpm setup`; CI is green including the Docker smoke test; a user logs in with MFA, opens a seeded patient, and the read appears in the audit log; the RLS test passes as `app_rw`; the PDF spike renders in the Repl; the PHI-guard test passes
M1 Programs and enrollment	3 weeks	Rule pack schema and five packs loaded; programs and templates admin; enrollment lifecycle with consent; care-plan templates and editor with required-element validation; sign and version; PDF export of care plan; configuration export and import	Admin creates "RTM – Spine", enrolls a patient, signs a plan, downloads the PDF; activation blocks RPM plus RTM for the same patient; a tenant's configuration exports from one tenant and imports into another

Milestone	Estimate	Epics (Linear issues are children of these)	Gate to pass
M2 Data in	3 weeks	Device catalog; device simulator; first cellular vendor adapter with webhook; observation pipeline; device-day roll-up; ePRO instruments (PHQ-9, GAD-7, pain NRS, ODI, PGIC) and scheduling; care-plan items to patient tasks; patient PWA (home, tasks, check-ins, devices); React Native patient app (onboarding, home, take reading, check-in) in Expo Go; one Bluetooth device model in an EAS development build	A simulator webhook creates a reading and a device day and shows on the readings tab; a patient completes a PHQ-9 on the PWA and on the app and the score appears
M3 Rules, work and GCP dry run	3 weeks	Rule engine with all seven kinds; template defaults; exceptions with dedupe and escalation; worklists; timer and time entries with the exclusion constraint; interactions; notes with templates and signing; secure messaging; notifications (console drivers); video (stub); clinician companion app; GCP dry run by DevOps using section 18 in a scratch project	A critical BP reading opens one exception, pages the on-call role (console), is resolved with a note, and the time is committed against RPM; the CI-built images start on Cloud Run against Cloud SQL and Memorystore and pass the smoke tests

Milestone	Estimate	Epics (Linear issues are children of these)	Gate to pass
M4 Billing	3 weeks	Period creation and close jobs; evidence engine for all five packs; concurrency enforcement; at-risk worklist; month-end report with approve, reject, export; per-patient audit pack PDF; reconciliation CI test; payer overrides; month generator	Seeded month closes; evidence matches hand-computed expectations for 20 fixture patients; the reconciliation test passes; a billing user approves and exports lines
M5 EHR, AI, hardening	3 weeks	FHIR R4 read adapter against fixtures and a sandbox; OmniMD adapter scoped and built to the agreed scope; patient matching; EHR tab; AI drivers and the five v1 capabilities with guardrails and evals; break-glass; HealthKit and Health Connect; offline sync hardening; performance budgets; runbooks; mobile store packages	Patient demographics and conditions pull from a FHIR sandbox; an AI monthly summary passes the numeric-fidelity check and is accepted into a note; on a seeded 5,000-enrollment dataset the at-risk worklist loads in under one second

Milestone	Estimate	Epics (Linear issues are children of these)	Gate to pass
M6 GCP production landing	3 weeks	Terraform for dev, staging, prod; Google Cloud BAA confirmed; Cloud SQL, Memorystore, Cloud Storage, KMS, Secret Manager, load balancer, Cloud Armor; Workload Identity Federation; deploy workflows enabled; production drivers wired and tested (`gcs`, `gcp-kms`, `vertex`, Twilio, Daily, device vendors); load test; penetration test on staging and fixes; backup and restore drill; dashboards and alerts; cutover rehearsal	Staging passes the full Playwright suite on GCP; the PHI guard allows `phi` only in prod; restore drill done; no open critical or high findings; load test meets budgets; DevOps signs off section 18
M7 Pilot and v1.0	2 weeks	Migration tooling for a legacy vendor's patients (CSV import of patients, enrollments, devices, consents with a validation report); tenant onboarding checklist; training materials; v1.0 release; both mobile apps submitted to the stores	First real tenant configured in prod; legacy patients imported with a validation report; first calendar month closed and reconciled with the customer's billing team

Phase 2 (after v1.0, separate plan): EHR document attach and discrete write-back after verification; outbound webhooks and partner API; reseller tenants; barrier clustering; UK region and NHS-specific requirements.

Definition of done for every issue

- Acceptance criteria met and demonstrated on the Replit build or on staging.
- Tests added at the right level (unit for logic, integration for API, Playwright for flows, Maestro for mobile) and passing in CI.

- No new lint, type or security warnings, and no Replit-only assumption (it works from a clean clone and in the Docker image).
- OpenAPI updated if a route changed; migration included if the schema changed; ADR added if a decision was made.
- PHI impact considered and noted on the PR.
- Documentation updated: runbook, admin guide, the GCP deployment guide or this spec as relevant.

Demo build for the current prospect

The 9 or 10 November demo precedes this build and runs on the existing Mahalo Care product. This plan is the production platform that the signed contract will be delivered on; the migration tooling in M7 is what moves the roughly 950 legacy RPM and CCM patients. Nothing promised in the demo may exceed what sections 6 to 11 describe. A Replit demo of the new build can show configuration, programs, the simulator and billing evidence with synthetic data well before M6, but it is never offered as the live service.

17. CLAUDE.md and replit.md — operating instructions

Copy the first block into `CLAUDE.md` at the repo root. It tells Claude Code how to work in this repository on every session. Keep it under 200 lines; detailed rules live in `docs/spec/`. The second block goes in `replit.md` so Replit Agent, if anyone uses it, follows the same rules.

```
# Mahalo Care — Claude Code operating instructions

## What this repo is
Mahalo Care: one configurable platform for RPM, RTM, CCM, PCM and BHI with
therapeutic templates.
The full specification is docs/spec/mahalo-care-spec.md. Read the relevant
section before starting any issue.
Platform only: we never deliver care, never make billing determinations,
never write to an EHR in v1.
We build and run on Replit with synthetic data only. Production runs on GCP.
The code must work in both with configuration changes only.

## How work starts
- Every task is a Linear issue in team CARE. If asked to do something with no
issue, create the issue first (title, acceptance criteria, milestone), then
work.
```

```
- Branch: `<LINEAR-KEY>-<slug>` from `main`. One issue per branch. One PR per branch, opened as draft.
- Before coding: read the spec section, read the module's README, run `pnpm test --filter <module>` to see the baseline.

## Commands
- `pnpm setup` — first-time setup (install, migrate, seed, start). Works in a fresh Repl and locally.
- `pnpm dev` — start api, web, patient PWA and worker. `pnpm dev --filter api` for one.
- `pnpm lint`, `pnpm typecheck`, `pnpm test`, `pnpm test:integration`, `pnpm test:e2e`.
- `pnpm db:generate` after schema changes; `pnpm db:migrate` (uses DATABASE_MIGRATE_URL); `pnpm db:seed` (demo patients, blocked when DATA_CLASS=phi); `pnpm db:seed --reference-only` (rule packs, templates, device catalog, instruments, roles: safe in every environment); `pnpm seed:month`; `pnpm tenant:create` (platform admin only); `pnpm smoke:isolation`.
- `pnpm contracts:generate` after any route change (regenerates OpenAPI and client).
- `pnpm rulepacks:validate` after any change under packages/rulepacks.
- `pnpm config:export` and `pnpm config:import` move tenant configuration (never patient data).
- `pnpm sim:device` runs the device simulator against the running API.

## Rules that are never broken
1. No PHI anywhere in this repo, Replit, logs, test fixtures, commit messages, PR descriptions or Linear issues. Use synthetic data from tools/seed.
2. Every tenant table has tenant_id, an RLS policy, and FORCE ROW LEVEL SECURITY. The app connects as app_rw (not the owner, no bypass). Every query goes through the request-scoped db with app.tenant_id set.
3. audit_log is append-only. Never write UPDATE or DELETE against it.
4. Only the billing module computes evidence; it never writes time_entry or device_day.
5. One time_entry belongs to one program. Never merge, split or round minutes.
6. AI output lands only in ai_draft. No code path writes AI text into clinical tables without a user acceptance event.
7. CPT codes, thresholds and concurrency rules change only in packages/rulepacks with a Linear issue labelled verify-cms linking the CMS source, and a clinical-ops reviewer on the PR.
8. No new UI component outside packages/ui or packages/ui-native. No inline
```

```
colors; use tokens.
9. Migrations are expand-then-contract and must pass up-down-up in CI.
10. Secrets only from Replit Secrets (test values) or Secret Manager. Never
commit a key.
11. Only packages/config reads process.env. Only packages/adapters imports
vendor SDKs. Add a driver, never a direct call.
12. No Replit-only service or SDK outside the storage adapter: no Replit
Auth, Replit DB, or Replit-specific URLs in code. Everything must also run in
the Docker images.
13. The PHI guard stays: DATA_CLASS=phi must refuse to start unless
APP_ENV=prod and every driver is a production driver.
14. Jobs are idempotent. Any process can restart at any time.

## Coding standards
- TypeScript strict, no `any`, no non-null assertions without a comment
explaining why.
- Zod schema first for every input and output; Fastify routes use the type
provider; export schemas from packages/contracts.
- Functions under 50 lines; modules export a small public surface; business
logic in services, not route handlers.
- Errors are typed (`AppError` with code and status); never throw strings.
- Tests next to code: `*.test.ts` unit, `*.int.test.ts` integration,
`e2e/*.spec.ts` Playwright. Integration tests read TEST_DB_MODE
(`testcontainers` in CI, `url` on Replit).
- Comments explain why, not what. No commented-out code.
- Dates: timestamptz in UTC; convert at the edge using the patient or tenant
timezone; never use `new Date()` in business logic — inject a clock.
- Services listen on HOST/PORT from config, log JSON to stdout, handle
SIGTERM, and expose /healthz and /readyz.

## Definition of done (check before marking a PR ready)
- Acceptance criteria met and shown in the PR (screenshots for UI, test
output for logic).
- Tests added; `pnpm lint typecheck test` green locally; CI green including
the Docker smoke test.
- OpenAPI regenerated if routes changed; migration included if schema
changed; ADR added in docs/adr if a decision was made.
- PR template filled, including PHI impact and the Replit-only-assumption
answer.
- Linear issue moved to In Review with a one-paragraph summary.

## When unsure
- Ambiguity in the spec: write the question in the Linear issue, pick the
```

```
safer option (more validation, less automation), and note the assumption in
the PR.
- Anything touching money, codes, consent or PHI: stop and ask in the issue
rather than guess.

## Do not
- Do not add dependencies without an ADR if they handle PHI, auth, crypto or
billing.
- Do not change CI, Terraform, Dockerfiles or CODEOWNERS in a feature PR;
open an infra issue.
- Do not run migrations against staging or prod from a local machine or from
Replit.
- Do not put real patient data, real phone numbers or live vendor keys into
Replit for any reason.
- Do not mark your own PR ready for review.
```

```
# replit.md — rules for anyone (or any agent) working in this Repl

This Repl is a build workspace for Mahalo Care. GitHub is the source of
truth.
Read CLAUDE.md first; every rule there applies here.

- Synthetic data only. Never enter real patient data, real contact details or
live credentials.
- Work on a branch named after the Linear issue. Never commit to main from
here.
- Start everything with `pnpm dev`. Do not edit .replit or replit.nix without
a Linear issue.
- Do not use Replit Auth, Replit Database or any Replit-only service. Use
Better Auth, Postgres, Redis and the adapters in packages/adapters.
- Read settings only through packages/config. Add new settings to the config
schema and to the table in the spec.
- The app must also run as the Docker images in docker/. If something works
only in this Repl, it is a bug.
- Database: connect as app_rw. Migrations run with DATABASE_MIGRATE_URL
through `pnpm db:migrate`.
- Do not generate your own login, user table or session handling.
- Run `pnpm lint typecheck test` before pushing.
```

Supporting files Claude Code should create in M0

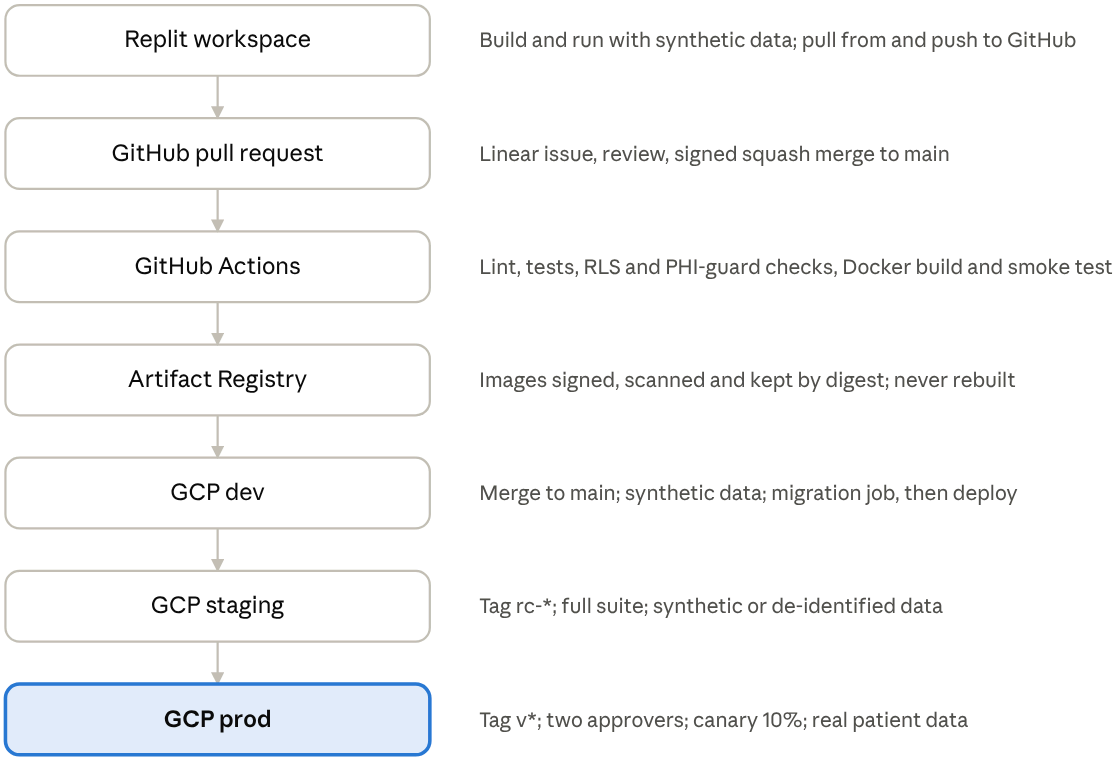
- `docs/spec/mahalo-care-spec.md` — this document exported as Markdown, with a header noting the Claude Doc link as the source of truth.

- `docs/gcp-deployment-guide.md` — section 18 of this document exported for DevOps, kept in sync.
- `docs/adr/0001-modular-monolith.md`, `0002-rls-tenancy.md`, `0003-append-only-audit.md`, `0004-rulepacks-as-data.md`, `0005-ai-draft-boundary.md`, `0006-replit-build-gcp-run.md`, `0007-adapters-and-phi-guard.md`.
- `apps/api/README.md` with the module map and the `authorize` and `audit` call patterns.
- `packages/rulepacks/README.md` explaining versioning, approval and the verify-cms label.
- `packages/adapters/README.md` listing every driver and the variable that selects it.
- `.github/PULL_REQUEST_TEMPLATE.md` with the fields in section 15.
- `docs/runbooks/period-close.md`, `deploy.md`, `rollback.md`, `restore.md`, `incident.md`, `replit-secrets.md`.

18. Replit to GCP deployment guide (for DevOps)

Deploying means building three images in CI, creating the GCP resources with Terraform, setting configuration and secrets, running the database migration job, and starting three Cloud Run services behind a load balancer. No application code changes. If a step needs a code change, that is a bug in the build and goes back to the build team as a Linear issue labelled `area:gcp`. The build team keeps this guide in `docs/gcp-deployment-guide.md` and updates it whenever the platform changes.

One image moves from CI to prod by digest; Replit never reaches production



promotion path · 7 stages, one image

The same image is promoted at every stage; only configuration and data change. Replit sits at the top as a workspace and never appears below GitHub.

What the build team hands over (checklist)

- A tagged commit on `main` and the matching image digests for `api`, `worker` and `web` in Artifact Registry.
- The `infra/terraform` modules and environment folders for dev, staging and prod.
- The environment variable contract (section 4) and the list of secrets with purpose, never values.
- `docs/runbooks/db-roles.sql` that creates the owner and `app_rw` roles, grants and extensions.
- Reference data seed (rule packs, default templates, device catalog, instruments, roles) and the tenant configuration JSON exported from Replit with `pnpm config:export`.
- The latest CI run: unit, integration, RLS isolation, PHI-guard, Docker smoke and Playwright results.

- A list of known deviations and open **VERIFY** items from section 19.

What changes between Replit and GCP

Concern	On Replit	On GCP	What you do
Processes	`pnpm dev` runs api, web, worker	Three Cloud Run services plus one job	Deploy the images; no code change
Database	Replit or external Postgres 16	Cloud SQL Postgres 16, private IP	Create roles with `db-roles.sql`; run the migration job
Redis	External Redis or in-Repl	Memorystore Redis 7 with `noeviction`	Set `REDIS_URL` secret
Files	`replit` or `local` storage driver	`gcs` driver, CMEK buckets	Set `STORAGE_DRIVER=gcs` and the bucket name
Encryption keys	Local key from a Replit Secret	Cloud KMS envelope keys	Set `CRYPTO_DRIVER=gcp-kms` and the key name
Secrets	Replit Secrets	Secret Manager, mounted as env vars	Create secrets, grant access per service account
AI	Anthropic API directly	Claude on Vertex AI	Set `AI_DRIVER=vertex`, project and location
Messaging and video	Console, stub or test accounts	Live Twilio, Postmark, Daily under BAAs	Set production drivers and secrets
Tenant resolution	`membership`	`subdomain`	Set `TENANT_RESOLUTION=subdomain`, wildcard DNS and certificate
Cron	BullMQ repeatable jobs in the worker	Same, worker never throttled	Min 1 instance, CPU always on

Concern	On Replit	On GCP	What you do
Public URL	Repl URL	Load balancer hostnames	Set `APP_BASE_URL` and `API_BASE_URL`
Data	Synthetic only	Real data only in prod	`DATA_CLASS=phi` only in prod

Image shapes

The real Dockerfiles live in `docker/` and are validated by CI on every pull request. These are the reference shapes, so DevOps knows what to expect.

```
# docker/api.Dockerfile (worker.Dockerfile is the same pattern on the
Playwright base image)
FROM node:22-bookworm-slim AS base
ENV PNPM_HOME=/pnpm PATH=/pnpm:$PATH
RUN corepack enable
WORKDIR /repo

FROM base AS prune
COPY . .
RUN pnpm dlx turbo prune --scope=api --docker

FROM base AS build
COPY --from=prune /repo/out/json/ .
RUN pnpm install --frozen-lockfile
COPY --from=prune /repo/out/full/ .
RUN pnpm turbo build --filter=api...
RUN pnpm --filter=api deploy --prod /out

FROM node:22-bookworm-slim AS run
ENV NODE_ENV=production HOST=0.0.0.0 TZ=UTC
WORKDIR /app
COPY --from=build /out .
USER node
EXPOSE 8080
CMD ["node", "dist/server.js"]
```

- `api` image: the migration job uses this same image with the command `node dist/migrate.js`, so schema and code always match.

- `worker` image: base is `mcr.microsoft.com/playwright` pinned to the same version as the Playwright package in the lockfile, so Chromium and fonts are present for PDF rendering. It exposes `/healthz` on `PORT` because Cloud Run needs an HTTP port.
- `web` image: static build served by an unprivileged nginx on port 8080 with single-page-app fallback and security headers. It contains no environment-specific value; it calls `/api` on its own origin, so the same image is promoted from dev to prod.
- All images run as a non-root user, have no shell tools beyond the base, and are scanned in Artifact Registry. Images are promoted by digest, never rebuilt.

Runtime settings per service (starting values; tune after the load test)

Service	Min and max instances	CPU and memory	Special settings
`api`	Min 1 in prod, 0 in dev; max set by load test	1 vCPU, 1 GiB to start	Concurrency 80, request timeout 60 s, ingress internal and load balancer only, VPC egress to the private network
`worker`	Min 1, max set by load test	2 vCPU, 2 GiB or more because of Chromium (**VERIFY** in the load test)	CPU always allocated (no throttling), no public ingress, VPC egress
`web`	Min 1 in prod, 0 in dev	0.25 vCPU, 256 MiB	Ingress internal and load balancer only
`migrate` job	Run once per release	1 vCPU, 1 GiB	Max retries 0, task timeout 10 minutes, uses only `DATABASE_MIGRATE_URL`

A Cloud Run worker pool may suit the worker better than a service if it is generally available when you build (**VERIFY**); the image and variables are the same either way.

Service accounts (least privilege)

Account	Can do
`sa-api`	Read its secrets; use the documents bucket (object user); use the KMS key (encrypt and decrypt); call Vertex AI; write logs and traces

Account	Can do
`sa-worker`	Same as `sa-api`
`sa-migrate`	Read only the `DATABASE_MIGRATE_URL` secret
`sa-deployer`	Assumed by GitHub Actions through Workload Identity Federation; deploy Cloud Run, push and read Artifact Registry, run the migration job; cannot read secret values

GCP resources (Terraform modules in `infra/terraform/modules`)

Module	Creates
`network`	VPC, subnets, private service access for Cloud SQL and Memorystore, Cloud NAT with a reserved static egress IP (some EHRs and vendors need an allow-listed IP; **VERIFY** with the first customer)
`sql`	Cloud SQL Postgres 16, HA in prod, private IP only, automated backups with 35-day retention, point-in-time recovery, CMEK, database flags for logging
`redis`	Memorystore Redis 7, AUTH and in-transit encryption on, `maxmemory-policy noeviction`, standard tier in prod
`storage`	Buckets for documents, exports and backups with CMEK, uniform access, versioning on documents, retention lock on the audit export bucket
`kms`	Key ring and keys for SQL, storage and field encryption, rotation schedule
`secrets`	Secret Manager secrets (shells; values are set by hand or by the secret owner)
`registry`	Artifact Registry repository with vulnerability scanning
`run`	Cloud Run services, the migration job, service accounts and IAM

Module	Creates
`lb`	Global external HTTPS load balancer, serverless network endpoint groups, URL map, managed certificates (wildcard through DNS authorization), Cloud Armor policy
`monitoring`	Dashboards, alert policies, uptime checks on `/readyz`, log-based metrics, log sink to the locked audit bucket
`iam`	Workload Identity Federation pool for GitHub, service accounts, organization policy constraints

Step-by-step runbook

1. **Confirm the prerequisites.** Google Cloud BAA executed on the organization; billing account; domain and DNS access for `care.mahalo.health` (or the chosen domain); BAAs with Twilio, the email provider, Daily and each device vendor; GitHub organization admin. Start Twilio 10DLC registration now, because it takes time.
2. **Create the projects.** `mahalo-care-dev`, `mahalo-care-stg`, `mahalo-care-prod` under a folder with HIPAA-aligned organization policies (no public buckets, no external IPs on databases, restricted regions, required CMEK). Link billing.
3. **Enable the APIs** in each project: Cloud Run, Cloud SQL Admin, Memorystore for Redis, Compute Engine, Service Networking, Secret Manager, Cloud KMS, Artifact Registry, Vertex AI, Certificate Manager, Cloud Logging, Cloud Monitoring, IAM and IAM Credentials.
4. **Create the Terraform state bucket** (versioned, CMEK) and run `terraform apply` for dev: `network`, `kms`, `registry`, `storage`, `sql`, `redis`, `secrets`, `iam`.
5. **Create database roles.** Connect through the Cloud SQL Auth Proxy from a bastion or Cloud Shell and run `db-roles.sql`: owner role for migrations, `app_rw` for the application, grants, default privileges, extensions. Confirm `app_rw` is not the owner and has no bypass-RLS right.
6. **Set the secrets.** `DATABASE_URL` (app role), `DATABASE_MIGRATE_URL` (owner role), `REDIS_URL`, `AUTH_SECRET`, and each provider secret. Replit values are never copied; generate new ones.
7. **Configure GitHub.** Set repository variables (`GCP_ENABLED=true`, project ids, region, Workload Identity provider, deployer account) and create GitHub Environments `dev`, `staging` and `production` with required reviewers (two for production).

8. **Build and push images.** Merge to `main` runs `images.yml`; confirm the three images and their scan results in Artifact Registry.
9. **Run the migration job** for dev, then the reference-data seed (`pnpm db:seed --reference-only`). Demo patients are never seeded where `DATA_CLASS=phi`.
10. **Deploy the services** to dev with `deploy-dev.yml`: `api`, `worker`, `web`, with the variables from section 4 set to the GCP column of the table above.
11. **Create the load balancer and DNS.** Path rules: `/api/*` and `/webhooks/*` go to `api`; everything else goes to `web`. Attach Cloud Armor (OWASP rules, rate limits on `/auth/*` and `/webhooks/*`). Point the wildcard and apex DNS records at the reserved global address; wait for certificates to become active.
12. **Create the first tenant.** Run `pnpm tenant:create` as a platform admin, import the configuration JSON with `pnpm config:import`, invite the first tenant admin, and complete MFA enrollment.
13. **Run the smoke tests** below on dev, then repeat steps 4 to 13 for staging and prod using the same images by digest.
14. **Set up observability.** Apply the `monitoring` module, route alerts to the on-call rotation, confirm an alert fires from a forced failure.
15. **Run the backup and restore drill** on staging: restore the latest backup to a new instance, point a test API at it, confirm the app reads it.
16. **Run the penetration test** against staging and close critical and high findings.
17. **Cut over the first tenant** (checklist below).

Smoke tests after every deploy

Check	How	Pass looks like
Health	`/healthz` and `/readyz` on each service	200 from all; `/readyz` confirms database and Redis
Sign in	Tenant subdomain, password plus MFA	Reaches the home page; audit log shows the login
Tenant isolation	`pnpm smoke:isolation` with two tenants	Tenant A cannot read tenant B through the app role
Audit is append-only	Try an UPDATE on `audit_log` as `app_rw`	The database refuses it

Check	How	Pass looks like
Webhook path	Signed test webhook from the simulator (staging) or a vendor test event (prod)	A reading appears and a device day is counted
Queue and cron	Check queue dashboard and repeatable jobs	Jobs registered once; a test job completes
PDF	Generate a care plan PDF	Opens correctly; object in the documents bucket with CMEK
Notifications	Send a test SMS and email to team recipients	Delivered; no clinical content in the text
Video	Open a test room	Both sides join
AI	Request a summary on synthetic staging data	Draft appears marked as draft; numeric check passes
PHI guard	Start a prod-config container with a non-production driver	It refuses to start
Edge	Send a known bad request through the load balancer	Cloud Armor blocks it; TLS certificate valid

Moving configuration, not data, from Replit

Export programs, templates, rules and care-plan templates on Replit with `pnpm config:export --tenant <slug>` and import them on GCP with `pnpm config:import`. Never move patient rows: Replit holds synthetic data only. Branding assets and consent texts come across with the configuration. Review the imported thresholds with the Clinical Safety Officer before the tenant goes live.

Mobile build profiles at cutover

Build the stores' release from the `production` profile in `eas.json`, which points at the production hostname. Do not reuse a `replit` profile build. TestFlight and Play internal builds use the `staging` profile against staging. Store submission waits until prod passes its smoke tests.

Rollback

- Application: route traffic back to the previous Cloud Run revision (one command or one click). Images are kept by digest, so the previous release is always available.

- Database: migrations are expand-then-contract, so the previous release works with the new schema. If a migration must be reversed, restore from point-in-time recovery to a new instance and repoint, following `docs/runbooks/restore.md`.
- Every production release has a documented rollback owner and a decision deadline before the canary reaches 100%.

Cutover checklist for the first live tenant

- [ ] Prod smoke tests pass and the penetration-test criticals are closed.
- [ ] BAAs on file for every vendor that will touch patient data.
- [ ] DNS, wildcard certificate and tenant subdomain live.
- [ ] Device vendor webhooks and the vendor accounts point at the production URL and secrets.
- [ ] EHR connection credentials and allow-listed egress IP confirmed with the customer's EHR owner.
- [ ] Twilio sender approved; email sender domain verified (SPF, DKIM, DMARC).
- [ ] Tenant configuration imported and reviewed; users invited with MFA.
- [ ] On-call rotation named, alert routing tested, incident runbook read.
- [ ] Backup and restore drill done on staging.
- [ ] Mobile apps built from the production profile and approved in the stores.
- [ ] Legacy data import (M7) rehearsed on staging with a validation report.

After cutover: retire Replit from the production path

Shut down any Replit deployment used for demos that pointed at real vendor accounts, rotate every secret that was ever stored in Replit, remove the external Redis and database used during the build, and keep the Repl only as a synthetic-data sandbox. Record the retirement as ISMS evidence.

Common problems when moving from Replit to GCP

Symptom	Likely cause	Fix
Service cannot reach database or Redis	Cloud Run has no VPC egress, or private service access is missing	Enable VPC egress on the service; check the `network` module
Revision fails to start	Process not listening on `PORT`, or bound to localhost	Bind `0.0.0.0` and read `PORT` from config

Symptom	Likely cause	Fix
Sign-in loops or CORS errors	`APP_BASE_URL`, `API_BASE_URL` or cookie domain wrong; web and api not on one origin	Fix the variables; keep path routing on the load balancer
Webhooks return 403 or time out	Cloud Armor rule, missing path rule, or signature secret mismatch	Check the policy log, the URL map and the secret
Jobs stop running	Worker CPU throttled or scaled to zero	Min 1 instance, CPU always allocated
Jobs run twice after a deploy	A job is not idempotent, or schedulers were registered with random ids	Fix the job; use stable scheduler ids
Every user sees no data, or sees everyone's	`app.tenant_id` not set, or app connects as owner	Check the app role and the request transaction
PDF generation fails or is killed	Missing fonts or libraries, or memory too low	Use the Playwright base image; raise worker memory
Queue jobs vanish	Redis eviction policy is not `noeviction`	Fix the Memorystore setting
Uploads over about 30 MiB fail	Cloud Run request size limit	Upload straight to Cloud Storage with signed URLs
First request after idle is slow	Scaled to zero	Set min instances above zero in prod
Times are off by hours	Process time zone not UTC	Set `TZ=UTC`; convert only at the edge

19. Open decisions and verification list

These items need a human answer before the milestone that depends on them. Each becomes a Linear issue labelled `blocked-customer`, `verify-cms` or `area:replit` in M0.

Decisions for Prasanna

- [ ] Confirm the builder: Claude Code in the Replit shell or locally as the default, with Replit Agent allowed only on feature branches for UI work under `replit.md`. Or Claude Code only.
- [ ] Confirm the stack choices that are new relative to the current Mahalo app: Better Auth for identity, Claude on Vertex AI as the production AI provider, Daily.co for video, Twilio for SMS. Each can be swapped behind its adapter; the decision only sets the M0 default.
- [ ] Confirm the Replit database choice: Replit's built-in Postgres or an external Postgres 16, and the Redis choice: external Redis or Redis inside the Repl.
- [ ] Confirm the first GCP region (`us-east4` proposed) and that the Google Cloud BAA is executed on the organization before any real data lands in prod.
- [ ] Confirm the repository name and GitHub organization, and that Linear team `CARE` can be created.
- [ ] Confirm that the GCP dry run in M3 goes ahead (recommended: it finds private networking, secrets and webhook problems early, at a small cost).
- [ ] Decide whether existing Mahalo Care code is reused for any module or this is a clean build on the new schema. This spec assumes a clean build with a data bridge for legacy patients in M7.

Verify on Replit in M0 (`area:replit`)

- [ ] Which external ports Replit exposes and how they map in `.replit`; whether one origin with a proxy works for web, api and webhooks.
- [ ] The Replit Postgres major version and that `pg_trgm`, `btree_gist` and row-level security with a non-owner role work as designed.
- [ ] That Redis (external or in the Repl) allows `noeviction` and runs BullMQ repeatable jobs reliably across Repl restarts.
- [ ] That Chromium from Nix renders a care plan PDF inside the Repl's memory limit.
- [ ] That the Expo Metro tunnel works from the Repl for Expo Go testing.
- [ ] Replit's resource limits for the full `pnpm dev` set of processes, and which checks must move to CI as a result.
- [ ] The Replit App Storage SDK and signed-URL behavior behind the storage adapter.

Verify with CMS sources before M4 (`verify-cms`)

- [ ] 2026 RPM codes 99445 and 99470 and their day and minute ranges.
- [ ] 2026 RTM codes 98979, 98984, 98985 and the MSK and respiratory supply splits.

- [ ] CCM, PCM and BHI minute thresholds and add-on caps as listed in section 6.
- [ ] Concurrency rules: RPM with RTM, CCM with PCM same practitioner, BHI with CCM, and the "one practitioner per period" rule for RPM and CCM.
- [ ] Whether patient-reported data satisfies RTM supply codes without a device, per code.
- [ ] Therapy modifiers (GP, GO, GN) for PT, OT and SLP billing RTM.
- [ ] Consent and initiating-visit requirements for CCM and PCM new patients.

Verify with the first customer before M2 and M5 (`blocked-customer`)

- [ ] Which cellular device vendor they use today for the roughly 600 RPM patients, and whether readings can be redirected or re-provisioned.
- [ ] OmniMD API access: who owns it on their side, the per-vendor connection process, which endpoints exist for demographics, problems, medications and document attach.
- [ ] The master or home location model they envision (section 14 table), and their place-of-service and group NPI setup per office.
- [ ] Their CCM second-condition mix, to seed the CCM template conditions list.
- [ ] Legacy export format from their current vendor for the migration tool in M7.
- [ ] Any EHR or vendor that needs an allow-listed egress IP.

The three capability claims from the deal handoff

The current demo runs on the existing Mahalo Care product; those three checks (PDF export of care plans and notes, one home billing location with cross-site access, structured care-plan templates for RTM, RPM and CCM) must be confirmed against that product by the Mahalo team before the call. In this specification all three are first-class requirements delivered in M1 and M0, so the production platform will hold them by design.

Assumptions this spec makes

- No 837 claim generation or clearinghouse integration in v1.
- US only in v1; UK and Singapore tenants follow in a later phase with region-specific rule packs.
- No research bridge in v1.
- Replit holds synthetic data only and is never a production or customer-facing live service.

- Estimates in section 16 are planning placeholders, not commitments; they are re-planned at the end of M0 with the actual team.